UBUNTU

CHEAT SHEETS

A Complete Reference for Ubuntu Linux Administration

Files & Permissions · Users & sudo · apt & snap · systemd · journalctl · Disks & LVM · Netplan · SSH · UFW · Nginx · Docker · Backups

49 REFERENCE SHEETS

CONTENTS

INTRODUCTION & INSTALL

01 Introduction & Release Cycle
02 Installing Ubuntu
03 First Boot Checklist
04 Getting Help

FILES & FILESYSTEM

05 Filesystem Layout
06 Navigating & Managing Files
07 Deleting Files Safely
08 Viewing & Editing Text
09 Permissions & Ownership
10 Finding Files & Archives

USERS & PRIVILEGES

11 Users & Groups
12 sudo & Root
13 umask, ACLs & Special Bits

PACKAGES

14 apt Basics
15 Repositories & PPAs
16 dpkg & .deb Files
17 Snap, Flatpak & Other Formats
18 Updates & Release Upgrades

PROCESSES & SERVICES

19 Processes
20 systemd & systemctl
21 Writing Service Units
22 Logs & journalctl
23 Scheduling Jobs

STORAGE & DISKS

24 Disks & Partitions
25 Filesystems & Mounting
26 LVM
27 ZFS & Software RAID

NETWORKING

28 Network Basics
29 Netplan
30 DNS & Hostnames
31 SSH
32 File Transfer

SECURITY

33 UFW Firewall
34 fail2ban & SSH Hardening
35 AppArmor
36 Ubuntu Pro & Security Updates

SERVER ROLES

37 Nginx & HTTPS
38 Docker on Ubuntu
39 File Sharing: Samba & NFS
40 VMs & System Containers
41 cloud-init & Autoinstall

DESKTOP

42 GNOME Desktop & Settings
43 Drivers & Hardware
44 Ubuntu on WSL

MONITOR & BACKUP

45 System Info & Performance
46 Boot, GRUB & Recovery
47 Backup & Restore

QUICK REFERENCE

48 Quick Reference — Files, Users & Packages
49 Quick Reference — Services, Storage & Network

Ubuntu Reference

Table of Contents

49 reference sheets · syntax · examples · notes

Sheet 01
Introduction & Release Cycle
Canonical · Debian roots · LTS vs interim · YY.MM versions · codenames · ESM · flavors

Sheet 02
Installing Ubuntu
ISO + bootable USB · desktop installer · Subiquity · LVM · encryption · OpenSSH · dual boot

Sheet 03
First Boot Checklist
apt update && apt upgrade · hostnamectl · timedatectl · locale · create a user · enable SSH · os-release

Sheet 04
Getting Help
man · --help · apropos · tldr · info · /usr/share/doc · Ask Ubuntu

Sheet 05
Filesystem Layout
/ · /etc · /home · /var · /usr · /opt · /tmp · /dev · /proc · merged /usr

Sheet 06
Navigating & Managing Files
pwd · cd · ls -lah · cp -r · mv · mkdir -p · touch · globbing

Sheet 07
Deleting Files Safely
rm / rm -r · rm -i · rmdir · trash-cli / gio trash · shred · no recycle bin

Sheet 08
Viewing & Editing Text
cat · less · head / tail -f · nano · vim basics · sudoedit · diff

Sheet 09
Permissions & Ownership
rwx · chmod 755 / u+x · chown user:group · -R · ls -l columns · stat

Sheet 10
Finding Files & Archives
find -name -mtime -exec · plocate · which / type · grep -r · tar -czf / -xzf · zip / unzip · zstd

Sheet 11
Users & Groups
adduser · usermod -aG · deluser · passwd · groups / id · /etc/passwd · /etc/group

Sheet 12
sudo & Root
sudo · sudo -i · sudo group · visudo · /etc/sudoers.d/ · sudo-rs · locked root

Sheet 13
umask, ACLs & Special Bits
umask · setfacl / getfacl · setuid · setgid on directories · sticky bit · chattr +i

Sheet 14
apt Basics
apt update · install · remove / purge · autoremove · search / show · list --installed · apt vs apt-get

Sheet 15
Repositories & PPAs
main / restricted / universe / multiverse · deb822 ubuntu.sources · add-apt-repository · signed-by · vendor repos

Sheet 16
dpkg & .deb Files
dpkg -i · apt install ./file.deb · dpkg -l / -L / -S · apt-mark hold · apt --fix-broken install

Sheet 17
Snap, Flatpak & Other Formats
snap install / refresh / remove · channels · --classic · snap list · Flatpak + Flathub · AppImage

Sheet 18
Updates & Release Upgrades
unattended-upgrades · dpkg-reconfigure · needrestart · reboot-required · do-release-upgrade · LTS to LTS

Sheet 19
Processes
ps aux · top / htop · kill / pkill / killall · signals · nice / renice · jobs / bg / fg · nohup

Sheet 20
systemd & systemctl
units · systemctl start / stop / restart / status · enable --now · is-active · list-units --failed · targets

Sheet 21
Writing Service Units
/etc/systemd/system/app.service · [Unit] / [Service] / [Install] · ExecStart · Restart=on-failure · User= · daemon-reload · systemctl edit

Sheet 22
Logs & journalctl
journalctl -u nginx · -f · --since · -p err · -b · /var/log · logrotate · dmesg

Sheet 23
Scheduling Jobs
crontab -e · cron syntax · /etc/cron.d · systemd timers · OnCalendar= · list-timers · at

Sheet 24
Disks & Partitions
lsblk · fdisk -l · parted · GPT vs MBR · /dev/sda vs /dev/nvme0n1 · blkid · wipefs

Sheet 25
Filesystems & Mounting
mkfs.ext4 · mount / umount · /etc/fstab · UUIDs · nofail · mount -a · USB drives

Sheet 26
LVM
PV / VG / LV · pvcreate · vgcreate · lvcreate · lvextend -r · resize2fs · snapshots

Sheet 27
ZFS & Software RAID
zpool create · mirrors / raidz · zfs create · snapshots · zpool status · mdadm RAID 1/5 · scrubs

Sheet 28
Network Basics
ip a · ip route · ss -tulpn · ping · curl / wget ·

Sheet 29
Netplan
/etc/netplan/*.yaml · DHCP vs static · addresses /

Sheet 30
DNS & Hostnames
hostnamectl set-hostname · /etc/hosts · systemd-

Sheet 01

Introduction & Release Cycle

Canonical · Debian roots · LTS vs interim · YY.MM versions · codenames · ESM · flavors

DESCRIPTION

Ubuntu is a Linux distribution built on **Debian** and published by **Canonical**. A new version ships every April and October, numbered **YY.MM**. April releases in even years are **LTS** (long-term support) and get years of security updates; the releases in between are **interim** releases with 9 months of support, used to try new features.

NOTES

- Current LTS: 26.04 "Resolute Raccoon"; previous: 24.04 "Noble Numbat"
- LTS: 5 years standard, up to 10+ with Ubuntu Pro (ESM)
- Same packages for desktop and server; the install differs
- Uses `apt` / `.deb` like Debian, plus Snap

WHICH UBUNTU AM I ON?

```
lsb_release -a
# Distributor ID: Ubuntu
# Description:    Ubuntu 26.04 LTS
# Release:        26.04
# Codename:       resolute

cat /etc/os-release          # same info, scriptable
. /etc/os-release && echo "$VERSION_CODENAME"   # resolute
uname -r                     # running kernel version
hostnamectl                  # OS, kernel, hardware summary
pro status                   # Ubuntu Pro / ESM coverage
```

CODE EXPLAINED

- → `lsb_release -a` — distro, version, codename
- → `/etc/os-release` — `KEY=value` file for scripts
- → codename — used in apt sources and docs
- → `uname -r` — kernel, separate from Ubuntu version
- → `pro status` — support and Ubuntu Pro state

OTHER DISTROS

- Debian — `cat /etc/debian_version`; stable every ~2 years
- Fedora/RHEL — `cat /etc/redhat-release`; RHEL 10-year life
- macOS — `sw_vers`

PREFER / AVOID

- Avoid interim releases on servers · Prefer the current LTS
- Avoid parsing `lsb_release` text · Prefer sourcing `/etc/os-release`

RECENT RELEASES

VERSION	CODENAME	TYPE
26.04	Resolute Raccoon	LTS
25.10	Questing Quokka	interim
24.04	Noble Numbat	LTS
22.04	Jammy Jellyfish	LTS
20.04	Focal Fossa	LTS (ESM)

FLAVORS & EDITIONS

NAME	WHAT
Ubuntu Desktop	GNOME desktop
Ubuntu Server	no GUI, Subiquity installer
Kubuntu / Xubuntu	KDE / Xfce desktops
Ubuntu Core	snap-only, IoT
Cloud images	AWS, Azure, GCP, LXD

⚠ COMMON MISTAKES

- Calling 25.10 "the latest" for a server: interim support ends after 9 months
- Confusing kernel version with Ubuntu version
- Following old guides blindly: tools and defaults change between LTS releases

✓ TIPS & BEST PRACTICES

- Check a guide's Ubuntu version before copying commands
- Plan LTS-to-LTS upgrades every 2–4 years (sheet 18)
- Ubuntu Pro is free for personal use on a few machines

QUICK SUMMARY

- `YY.MM` — April/October releases
- LTS — even-year April, long support
- `/etc/os-release` — version for scripts
- flavors — same base, different desktops

Installing Ubuntu

ISO + bootable USB · desktop installer · Subiquity · LVM · encryption · OpenSSH · dual boot

DESCRIPTION

Download the ISO from ubuntu.com, write it to a USB stick, and boot from it. **Desktop** has a graphical installer with a live session to test hardware first. **Server** uses the text-based **Subiquity** installer, which offers LVM, disk encryption, an OpenSSH server and SSH key import from GitHub or Launchpad.

NOTES

- Verify the ISO checksum before writing it
- Server installer: tick "Install OpenSSH server"
- Default server layout: LVM using about half the disk
- Dual boot: shrink Windows first, disable Fast Startup

⚠ DANGER CHECK

- Writing the ISO erases the whole USB stick
- "Erase disk and install" wipes every partition on that drive
- Check the target disk name (`lsblk`) before picking it

MAKE A BOOTABLE USB (LINUX/MACOS)

```
# verify download
sha256sum ubuntu-26.04-live-server-amd64.iso
# compare with SHA256SUMS from releases.ubuntu.com

lsblk                                  # find the USB, e.g. /dev/sdb
sudo dd if=ubuntu-26.04-live-server-amd64.iso of=/dev/sdb \
        bs=4M status=progress conv=fsync

# macOS: diskutil list → /dev/disk4
sudo dd if=ubuntu.iso of=/dev/rdisk4 bs=4m
# or use balenaEtcher / Rufus (Windows) / Startup Disk Creator
```

CODE EXPLAINED

- → `sha256sum` — confirm the ISO isn't corrupted
- → `lsblk` — identify the USB by size
- → `of=/dev/sdb` — the whole device, not `sdb1`
- → `conv=fsync` — flush before finishing
- → GUI tools do the same, with safety checks

OTHER DISTROS

- Debian — netinst ISO, text or graphical installer
- Fedora — Anaconda installer, Fedora Media Writer
- macOS — Recovery mode reinstall

PREFER / AVOID

- Avoid skipping LVM on servers · Prefer LVM for easy resizing
- Avoid password-only SSH from day one · Prefer importing SSH keys in the installer

INSTALLER CHOICES

OPTION	PICK WHEN
Use entire disk + LVM	most servers
Encrypt with LUKS	laptops, sensitive data
Custom storage	RAID, separate /home
Minimized server	containers, small VMs
Install OpenSSH	any remote server

HARDWARE NOTES

ITEM	CHECK
UEFI / Secure Boot	supported; MOK enroll for drivers
RAM	server 1.5 GB+, desktop 6 GB+
Disk	server 5 GB+, desktop 25 GB+
Old Macs	may need Broadcom Wi-Fi driver

⚠ COMMON MISTAKES

- Writing the ISO to a partition (`/dev/sdb1`) instead of the device
- Choosing the wrong disk in "erase disk" on multi-drive machines
- Forgetting the LVM default leaves half the disk unallocated

✓ TIPS & BEST PRACTICES

- Grow the root LV later with `lvextend -r` (sheet 26)
- Use autoinstall YAML for repeatable installs (sheet 41)
- Test the live session on desktops before installing

QUICK SUMMARY

- ISO + checksum + USB — install media
- Subiquity — server installer
- LVM + OpenSSH — sensible server defaults
- double-check the target disk

First Boot Checklist

apt update && apt upgrade · hostnamectl · timedatectl · locale · create a user · enable SSH · os-release

DESCRIPTION

Right after installing, bring the system up to date, set its **name**, **time zone** and **locale**, make sure you have a non-root user with **sudo**, and confirm **SSH** works with a key. Doing this first avoids odd problems later, like wrong timestamps in logs or a server you can only reach from its console.

NOTES

- `apt update` refreshes lists; `apt upgrade` installs updates
- Root is locked by default; your user uses `sudo`
- Time sync is on by default; check it once
- Reboot if `/var/run/reboot-required` exists

FIRST 10 MINUTES

```
sudo apt update && sudo apt full-upgrade -y
sudo hostnamectl set-hostname homeserver
sudo timedatectl set-timezone America/Chicago
timedatectl                              # NTP synchronized: yes
sudo localectl set-locale LANG=en_US.UTF-8
sudo adduser bob && sudo usermod -aG sudo bob
sudo apt install -y openssh-server
systemctl status ssh                     # active (running)
# from your laptop:
ssh-copy-id bob@homeserver.local
ls /var/run/reboot-required && sudo reboot
```

CODE EXPLAINED

- → `full-upgrade` — also handles changed dependencies
- → `hostnamectl set-hostname` — permanent name
- → `timedatectl set-timezone` — local time for logs/cron
- → `usermod -aG sudo` — add to the sudo group
- → `ssh-copy-id` — install your public key

OTHER DISTROS

- Debian — same commands; root may be enabled if you set a password
- Fedora — `sudo dnf upgrade`, user in `wheel` group
- macOS — `softwareupdate -ia`, `scutil --set HostName`

PREFER / AVOID

- Avoid working as root · Prefer a sudo user
- Avoid leaving updates for later · Prefer patching on day one

QUICK FACTS

COMMAND	SHOWS
`hostnamectl`	name, OS, kernel
`timedatectl`	time zone, NTP
`localectl`	locale, keymap
`ip -br a`	IP addresses
`df -h /`	root disk space
`free -h`	memory

DAY-ONE PACKAGES

PACKAGE	WHY
`openssh-server`	remote access
`unattended-upgrades`	auto security fixes (18)
`ufw`	firewall (33)
`htop / btop`	monitoring
`curl, git, vim`	basics

⚠ COMMON MISTAKES

- Changing the hostname without updating `/etc/hosts` (sudo warns "unable to resolve host")
- Leaving the time zone as UTC and misreading cron schedules
- Closing the console before confirming SSH key login works

✓ TIPS & BEST PRACTICES

- Use `ip -br a` for a one-line-per-interface view
- Set up `unattended-upgrades` before you forget (sheet 18)
- Write down what you changed; future you will thank you

QUICK SUMMARY

- `apt update && apt full-upgrade` — patch
- `hostnamectl` / `timedatectl` — identity and time
- sudo user + SSH key — safe access
- reboot if required

Getting Help

DESCRIPTION

Almost every command documents itself. **man** pages are the full reference, **--help** gives a quick summary, and **tldr** shows the handful of examples you usually need. When you don't know the command name, **apropos** searches man page descriptions. Ubuntu-specific questions are well covered on Ask Ubuntu and Ubuntu Discourse.

NOTES

- Minimized server images strip man pages: `unminimize` restores them
- Man sections: 1 commands, 5 file formats, 8 admin commands
- `man 5 crontab` ≠ `man crontab`
- Package docs live in `/usr/share/doc/<pkg>/`

FINDING ANSWERS

```
man ls                          # full manual; / to search, q to quit
man 5 sshd_config               # section 5: config file format
ls --help | less                # quick option summary
apropos "disk usage"            # du (1), df (1), ...
whatis rsync                    # one-line description
type ll                         # alias? builtin? file?
help cd                         # bash builtins use `help`
sudo apt install tldr && tldr tar    # example-first cheat sheet
ls /usr/share/doc/nginx/        # package docs and changelogs
```

CODE EXPLAINED

- → `man <cmd>` — the reference; `/word` searches
- → `man 5 <file>` — config file docs
- → `apropos` — search by keyword
- → `type` — what a name actually runs
- → `help` — docs for shell builtins

OTHER DISTROS

- Debian/Fedora — identical man/apropos/tldr
- macOS — BSD man pages; options differ from GNU

PREFER / AVOID

- Avoid copying commands you don't understand · Prefer `man` / `tldr` first
- Avoid googling flags for builtins · Prefer `help <builtin>`

MAN SECTIONS

#	CONTENTS
1	user commands
2 / 3	system calls / library
5	file formats (fstab, sudoers)
7	overviews (regex, signal)
8	admin commands (mount, ufw)

INSIDE MAN / LESS

KEY	DOES
/text	search forward
n / N	next / previous match
g / G	top / bottom
space / b	page down / up
q	quit

⚠ COMMON MISTAKES

- `man cd` finds nothing: it's a shell builtin; use `help cd`
- Trusting a 2014 forum answer for a 2026 release
- Ignoring `--help` because the man page is long

✓ TIPS & BEST PRACTICES

- Run `unminimize` on minimized servers you log into
- Use `man -k regex` (same as apropos) with patterns
- Read `/usr/share/doc/<pkg>/README.Debian` for Ubuntu-specific notes

QUICK SUMMARY

- `man` — full reference
- `--help` / `tldr` — quick usage
- `apropos` — find the command
- `help` — shell builtins

Filesystem Layout

DESCRIPTION

Linux has a single directory tree starting at `/`; disks are attached (mounted) somewhere in it. Ubuntu follows the **Filesystem Hierarchy Standard**: configuration in `/etc`, user files in `/home`, changing data and logs in `/var`, installed software in `/usr`. Knowing where things live is half of system administration.

NOTES

- Everything is a file, including devices (`/dev/sda`)
- `/proc` and `/sys` are virtual: kernel info, not disk files
- Merged `/usr` : `/bin` → `/usr/bin` symlinks
- Paths are case-sensitive: `Docs` ≠ `docs`

LOOK AROUND

```
ls /                          # top-level directories
ls /etc | head                # system-wide config files
ls -l /bin                    # /bin -> usr/bin (merged /usr)
du -sh /var/log               # how big are the logs?
cat /proc/cpuinfo | grep "model name" | head -1
cat /proc/meminfo | head -3
ls /dev/sd* /dev/nvme*        # disks
findmnt /                     # which device is mounted at /
echo $HOME                    # /home/bob
ls -a ~                       # dotfiles: .bashrc, .ssh/
```

CODE EXPLAINED

- → `/etc` — text config files, edited with sudo
- → merged `/usr` — old paths still work via symlinks
- → `/proc/*` — live kernel data as files
- → `findmnt` — what's mounted where
- → `~` — your home directory

OTHER DISTROS

- Debian/Fedora — same FHS layout, merged /usr too
- macOS — `/Users` not `/home` , `/Applications` , `/Library`

PREFER / AVOID

- Avoid putting your own apps in `/usr/bin` · Prefer `/usr/local` or `/opt`
- Avoid storing data in `/tmp` · Prefer `/var/lib/<app>` or `/srv`

KEY DIRECTORIES

PATH	HOLDS
`/etc`	configuration
`/home`	user files
`/var`	logs, caches, DBs, mail
`/usr`	installed programs, libraries
`/opt`	third-party apps
`/srv`	data served by this host

SPECIAL DIRECTORIES

PATH	HOLDS
`/tmp`	temp, cleared on boot
`/dev`	device files
`/proc, /sys`	kernel info
`/run`	runtime state (tmpfs)
`/boot`	kernels, GRUB
`/mnt, /media`	manual / auto mounts

⚠ COMMON MISTAKES

- Filling `/var` with logs or Docker images until services fail
- Editing files under `/usr/lib` that get overwritten by updates
- Assuming `/tmp` survives a reboot

✓ TIPS & BEST PRACTICES

- Override vendor configs in `/etc` , never in `/usr/lib`
- Put a separate disk at `/srv` or `/var/lib/docker` for big data
- Use `ncdu /` to find what's eating space (sheet 45)

QUICK SUMMARY

- `/` — one tree; disks mount into it
- `/etc` config, `/var` data, `/usr` programs
- `/proc` , `/sys` — virtual kernel files
- `/opt` , `/usr/local` — your installs

Sheet 06

Navigating & Managing Files

pwd · cd · ls -lah · cp -r · mv · mkdir -p · touch · globbing

DESCRIPTION

Daily file work is a small set of commands: **cd** to move, **ls** to look, **cp**, **mv** and **mkdir** to organize. **Globs** like `*.log` let the shell expand patterns into file lists before the command runs. Ubuntu 25.10+ uses the Rust **uutils** coreutils by default; everyday flags behave the same.

NOTES

- `cd` alone or `cd ~` → home; `cd -` → previous dir
- `.` = current dir, `..` = parent
- Hidden files start with `.`; `ls -a` shows them
- Quote names with spaces: `"My Files"`

EVERYDAY COMMANDS

```
pwd                          # /home/bob
cd /var/log && cd -          # go and come back
ls -lah                      # long, all, human sizes
ls -lt | head                # newest first
mkdir -p projects/api/src    # create parents as needed
touch notes.txt              # create empty / update time
cp report.pdf backup/        # copy file into dir
cp -a site/ site.bak/        # copy dir, keep perms + times
mv draft.md final.md         # rename
mv *.jpg ~/Pictures/         # glob: all .jpg files
cp -i a.txt b.txt            # ask before overwrite
ls ~/projects/{api,web}      # brace expansion
```

CODE EXPLAINED

- → `ls -lah` — permissions, owner, size, date
- → `mkdir -p` — no error if it exists
- → `cp -a` — archive mode for directories
- → `mv` — move and rename are the same thing
- → `{a,b}` — expands to two paths

OTHER DISTROS

- Debian/Fedora — GNU coreutils, same flags
- macOS — BSD `ls` / `cp`; `cp -a` OK, some GNU flags missing

PREFER / AVOID

- Avoid `cp -r` for backups · Prefer `cp -a` (keeps metadata)
- Avoid unquoted variables in paths · Prefer `"$dir"`

GLOBS

PATTERN	MATCHES
`*`	anything
`?`	one character
`[abc]`	one of a, b, c
`**/*.py`	recursive (bash: shopt -s globstar)
`{jpg,png}`	brace list

LS FLAGS

FLAG	EFFECT
`-l`	long format
`-a`	include hidden
`-h`	human sizes
`-t / -S`	sort by time / size
`-r`	reverse
`-R`	recursive

⚠ COMMON MISTAKES

- `mv file dir` when `dir` doesn't exist renames the file to `dir`
- `cp` silently overwrites; use `-i` or `-n` for safety
- Unmatched glob passed literally (`*.log` when none exist)

✓ TIPS & BEST PRACTICES

- Tab-complete paths; double Tab lists options
- Use `tree -L 2` (apt install tree) for a quick overview
- Add `alias ll='ls -lah'` to `~/.bashrc`

QUICK SUMMARY

- `cd`, `ls`, `pwd` — move and look
- `cp -a`, `mv`, `mkdir -p` — organize
- globs — shell expands patterns
- quote paths with spaces

Sheet 07

Deleting Files Safely

rm / rm -r · rm -i · rmdir · trash-cli / gio trash · shred · no recycle bin

DESCRIPTION

On the command line, **`rm` deletes immediately**: there is no recycle bin and no undo. `rm -r` removes whole directory trees. Build safe habits: look before you delete, use `-i` or a trash tool for interactive work, and never combine `sudo`, `-rf` and a variable you haven't checked.

NOTES

- Deleted ≠ gone from disk, but recovery is unreliable
- `rmdir` only removes empty directories (safe)
- Desktop file manager uses Trash; `rm` doesn't
- `--preserve-root` is on by default (`rm -rf /` refused)

⚠ DANGER CHECK

- `sudo rm -rf "$DIR"/*` with an empty `$DIR` becomes `rm -rf /*`
- A stray space: `rm -rf / tmp/x` deletes from `/`
- Use `ls` with the exact same arguments first

DELETE WITH CARE

```
rm old.log                          # gone, no undo
rm -i *.tmp                         # confirm each file
rm -I big-dir/*                     # one prompt for many files
rmdir empty-folder                  # fails if not empty: good
rm -r build/                        # directory tree
ls build/*.o && rm build/*.o        # preview, then delete

sudo apt install trash-cli
trash-put report.docx               # recoverable
trash-list && trash-restore
gio trash notes.txt                 # GNOME trash, same idea

: "${DIR:?DIR is not set}"          # scripts: stop if empty
rm -rf -- "$DIR/cache"
```

CODE EXPLAINED

- → `rm -i` / `-I` — prompts before deleting
- → `rmdir` — only empty dirs, a built-in safety
- → `ls` first — same glob, see what matches
- → `trash-put` — move to Trash instead
- → `${DIR:?}` — script aborts if `DIR` is empty

OTHER DISTROS

- Debian/Fedora — same `rm`; `trash-cli` packaged too
- macOS — `rm` the same; `trash` via Homebrew or Finder

PREFER / AVOID

- Avoid `rm -rf` as muscle memory · Prefer `rm -r` and read prompts
- Avoid `rm -rf $VAR/` unquoted · Prefer `"${VAR:?}"`

RM OPTIONS

FLAG	EFFECT
`-r`	recursive
`-f`	force, no prompts/errors
`-i`	prompt every file
`-I`	prompt once for > 3 files
`-v`	print each removal
`--`	end of options (names starting with -)

SAFER ALTERNATIVES

TOOL	USE
`trash-put`	recoverable delete
`find ... -delete`	after a dry run without -delete
`mv to ~/old/`	delete later
`shred -u file`	overwrite then delete (HDD)

⚠ COMMON MISTAKES

- Running `rm -rf` in the wrong directory: check `pwd` first
- Expecting `shred` to securely wipe SSDs: use full-disk encryption
- Deleting a log a process still writes to: space isn't freed until it restarts

✓ TIPS & BEST PRACTICES

- Use `find . -name '*.tmp'` first, then add `-delete`
- Truncate busy logs with `: > app.log` instead of deleting
- Keep backups (sheet 47); they're the real undo

QUICK SUMMARY

- `rm` — immediate, no undo
- `-i` / `trash-put` — safety nets
- preview with `ls` / `find`
- `${VAR:?}` — guard script deletes

Viewing & Editing Text

cat · less · head / tail -f · nano · vim basics · sudoedit · diff

DESCRIPTION

Linux configuration is plain text, so reading and editing files is a core skill. **less** pages through big files, **tail -f** follows logs live, **nano** is the friendly editor installed everywhere, and **vim** is worth knowing at least well enough to save and quit. Use **sudoedit** to edit system files safely.

NOTES

- `nano` is the default `EDITOR` on Ubuntu
- `sudo update-alternatives --config editor` changes it
- `sudoedit` edits a temp copy as you, then installs it
- `less +F` = follow mode like `tail -f`

READ, FOLLOW, EDIT

```
cat /etc/hostname                    # small files
less /var/log/syslog                 # page; /search, q quit
head -n 20 access.log                # first 20 lines
tail -n 50 access.log                # last 50 lines
tail -f /var/log/nginx/error.log     # follow live (Ctrl+C)
grep -n "error" app.log | less       # search with line numbers
nano ~/.bashrc                       # Ctrl+O save, Ctrl+X exit
sudoedit /etc/ssh/sshd_config        # safe root edit
vim notes.md                         # i insert, Esc, :wq save+quit
diff -u nginx.conf nginx.conf.bak    # what changed?
```

CODE EXPLAINED

- → `less` — doesn't load the whole file
- → `tail -f` — watch new lines appear
- → `nano` — shortcuts shown at the bottom
- → `sudoedit` — your editor settings, root file
- → `diff -u` — unified diff, like git

OTHER DISTROS

- Debian — same; `vi` may be `vim-tiny`
- Fedora — `vi` default `EDITOR`; nano installed
- macOS — `nano` is a `pico` alias in recent versions

PREFER / AVOID

- Avoid `sudo nano` habitually · Prefer `sudoedit`
- Avoid `cat bigfile` · Prefer `less` or `head`

VIM SURVIVAL

KEYS	DOES
`i`	insert mode
`Esc`	back to normal mode
`:w / :q`	save / quit
`:wq or ZZ`	save and quit
`:q!`	quit without saving
`/text`	search

NANO KEYS

KEYS	DOES
`Ctrl+O`	save
`Ctrl+X`	exit
`Ctrl+W`	search
`Ctrl+K / Ctrl+U`	cut / paste line
`Alt+U`	undo
`Ctrl+_`	go to line

⚠ COMMON MISTAKES

- Getting stuck in vim: press Esc then type `:q!`
- Editing a config without a backup: `sudo cp file file.bak` first
- `tail -f` on a rotated log stops showing lines; use `tail -F`

✓ TIPS & BEST PRACTICES

- Validate configs before restarting: `nginx -t`, `sshd -t`, `visudo -c`
- Use `journalctl -f` for systemd services (sheet 22)
- Set `export EDITOR=nano` (or vim) in `~/.bashrc`

QUICK SUMMARY

- `less` / `head` / `tail` — read
- `tail -F` — follow logs
- `nano` / `vim` — edit
- `sudoedit` — safe system edits

Permissions & Ownership

DESCRIPTION

Every file has an **owner**, a **group**, and three sets of **permissions**: for the owner (u), the group (g) and everyone else (o). Each set allows **read**, **write** and/or **execute**. On directories, execute means "may enter". Change modes with **chmod** and owners with **chown**.

NOTES

- Octal: r=4, w=2, x=1 → `7` =rwx, `5` =r-x, `4` =r--
- Only root (or sudo) can change a file's owner
- Scripts need `x` to run as `./script.sh`
- New files get permissions from `umask` (sheet 13)

⚠ DANGER CHECK

- `chmod -R 777` makes everything world-writable: never a fix
- `chown -R` on `/` or `/usr` can break sudo and the whole system
- Recursive changes on the wrong path are hard to undo

READ AND CHANGE PERMISSIONS

```
ls -l deploy.sh
# -rwxr-x--- 1 bob devs 812 Nov 13 09:10 deploy.sh
#  └┬┘└┬┘└┬┘   owner group

chmod u+x deploy.sh           # add execute for owner
chmod go-w notes.txt          # remove write for group/other
chmod 644 index.html          # rw-r--r--
chmod 755 /srv/site           # rwxr-xr-x (dir: enter + list)
chmod 600 ~/.ssh/id_ed25519   # private key: owner only
sudo chown www-data:www-data /srv/site/uploads
sudo chown -R bob: ~/projects      # bob + bob's group
stat -c '%A %a %U:%G %n' deploy.sh  # -rwxr-x--- 750 bob:devs
```

CODE EXPLAINED

- → first char — `-` file, `d` dir, `l` link
- → `u+x`, `go-w` — symbolic changes
- → `644` / `755` — octal modes for files / dirs
- → `chown user:group` — change both at once
- → `stat -c` — show mode in both forms

OTHER DISTROS

- Debian/Fedora — same tools; Fedora adds SELinux labels (`ls -Z`)
- macOS — same model plus ACLs; `chmod +a` syntax differs

PREFER / AVOID

- Avoid `chmod 777` to "fix" access · Prefer the right owner/group
- Avoid `chmod -R 755` on files · Prefer `find -type d` / `-type f` separately

COMMON MODES

MODE	USE
644	normal files
755	dirs, executables
600	secrets, SSH keys
700	private dirs (~/.ssh)
640	config readable by group
775	shared group dir

MEANING ON DIRECTORIES

BIT	ALLOWS
r	list names
w	create, delete, rename inside
x	enter, access files by name
r without x	names only, no access

⚠ COMMON MISTAKES

- `chmod -R 755 dir` makes every file executable
- SSH refuses keys when `~/.ssh` or the key is group/world-writable
- Changing perms instead of adding the user to the right group

✓ TIPS & BEST PRACTICES

- `find dir -type d -exec chmod 755 {} +` and `-type f ... 644`
- Use groups for shared access: `usermod -aG devs bob`
- `namei -l /path/to/file` shows perms on every parent dir

QUICK SUMMARY

- owner / group / other × r w x
- `chmod` — symbolic or octal
- `chown user:group` — ownership
- never `777`; fix ownership instead

Finding Files & Archives

DESCRIPTION

find searches the live filesystem by name, size, age, type or owner and can act on results. **plocate** answers "where is that file?" instantly from an index. **grep -r** searches inside files. **tar** bundles directories into one archive, compressed with gzip, xz or **zstd**.

NOTES

- `plocate` index updates daily; `sudo updatedb` to refresh
- `find` options go after the path
- `tar` flags: c create, x extract, t list, f file
- `.tar.zst` is fast and small; Ubuntu packages use zstd

SEARCH & ARCHIVE

```
find ~ -name "*.pdf" -mtime -7          # PDFs changed in 7 days
find /var/log -size +100M               # big files
find . -type f -name "*.tmp" -delete    # delete matches (careful)
find /srv -user www-data -type d        # dirs owned by www-data
plocate nginx.conf                      # instant, from index
which python3 && type -a python3        # what runs?
grep -rn "listen 443" /etc/nginx/       # search contents

tar -czf site-2026-11-13.tar.gz /srv/site   # create .tar.gz
tar -tzf site-2026-11-13.tar.gz | head      # list
tar -xzf site.tar.gz -C /tmp/restore        # extract to dir
tar --zstd -cf logs.tar.zst /var/log/app    # zstd archive
zip -r photos.zip Pictures/ && unzip -l photos.zip
```

CODE EXPLAINED

- → `-mtime -7` — modified less than 7 days ago
- → `-size +100M` — bigger than 100 MB
- → `-delete` — acts on matches; run without it first
- → `tar -czf` — create, gzip, to file
- → `-C dir` — extract somewhere else

OTHER DISTROS

- Debian — `plocate` too; Fedora uses `plocate` / `mlocate`
- macOS — `mdfind` (Spotlight); BSD `find` / `tar` mostly compatible

PREFER / AVOID

- Avoid `find ... -exec rm {} \;` · Prefer `-delete` or `-exec ... {} +`
- Avoid `tar` with absolute paths you'll extract over live files · Prefer `-C` to a temp dir

FIND TESTS

TEST	MATCHES
`-name / -iname`	name (case-insensitive)
`-type f / d / l`	file / dir / link
`-mtime -N / +N`	age in days
`-size +10M`	size
`-user / -group`	owner
`-perm -o+w`	world-writable

TAR CHEATS

COMMAND	DOES
`tar -czf a.tgz dir`	create gzip
`tar -cJf a.txz dir`	create xz
`tar --zstd -cf a.tar.zst dir`	create zstd
`tar -tf a.tgz`	list
`tar -xf a.tgz`	extract (auto-detects)
`--exclude='*.log'`	skip files

⚠ COMMON MISTAKES

- Putting `-delete` before the tests: `find . -delete -name x` deletes everything
- Forgetting `-f` in `tar`: it tries to use a tape device
- `plocate` doesn't find new files until `updatedb` runs

✓ TIPS & BEST PRACTICES

- Install `fd-find` and `ripgrep` for faster, friendlier search
- `tar -xf` detects compression automatically
- Add dates to archive names for easy sorting

QUICK SUMMARY

- `find` — live search + actions
- `plocate` — instant indexed lookup
- `grep -r` — search contents
- `tar -czf` / `-xf` — archive and extract

Users & Groups

DESCRIPTION

Every process runs as a **user**, and access is granted through **groups**. Ubuntu's **adduser** is the friendly way to create users (it makes the home directory and asks for a password); `usermod -aG` adds a user to more groups. Service accounts like `www-data` have no login shell and exist only to run software.

NOTES

- Group changes apply at next login (or `newgrp`)
- Human users start at UID 1000
- `/etc/shadow` holds password hashes (root only)
- `useradd` is the low-level tool; prefer `adduser`

MANAGE USERS

```
sudo adduser alice                      # interactive: password, name
sudo usermod -aG sudo,docker alice      # append to groups
groups alice                            # alice : alice sudo docker
id alice                                # uid=1001(alice) gid=1001 ...
sudo passwd alice                       # set/reset password
sudo passwd -l alice                    # lock account
sudo chage -l alice                     # password aging info
sudo addgroup devs && sudo adduser alice devs
sudo adduser --system --group --no-create-home myapp   # service user
sudo deluser --remove-home bob          # delete user + home
who && last -n 5                        # logged in now / recent logins
```

CODE EXPLAINED

- → `adduser` — creates user, group, home, prompts
- → `usermod -aG` — **a**ppend; without `-a` replaces groups
- → `passwd -l` — lock without deleting
- → `--system` — no-login account for a service
- → `last` — login history

OTHER DISTROS

- Debian — same `adduser`; Fedora — `useradd` + `passwd`, `wheel` group
- macOS — `sysadminctl -addUser`, System Settings → Users

PREFER / AVOID

- Avoid `usermod -G` without `-a` · Prefer `usermod -aG`
- Avoid running services as your user · Prefer a `--system` user

KEY FILES

FILE	HOLDS
/etc/passwd	users, UID, home, shell
/etc/shadow	password hashes
/etc/group	groups and members
/etc/skel/	template for new homes
/etc/login.defs	UID ranges, defaults

USEFUL GROUPS

GROUP	GRANTS
sudo	admin via sudo
adm	read /var/log
docker	use Docker (root-equivalent)
www-data	web server files
dialout	serial ports
lxd	manage LXD

⚠ COMMON MISTAKES

- `usermod -G docker bob` drops bob from `sudo`; always use `-a`
- Expecting new group access in the same shell session
- Adding users to `docker` without realizing it's root-equivalent

✓ TIPS & BEST PRACTICES

- `getent passwd alice` works with LDAP too
- Use `sudo -iu alice` to test as another user
- Remove ex-users promptly and lock shared accounts

QUICK SUMMARY

- `adduser` — create users
- `usermod -aG` — add to groups
- `id` / `groups` — check membership
- `--system` — service accounts

sudo & Root

DESCRIPTION

Ubuntu locks the **root** account and gives admins **sudo**, which runs one command as root after you type *your* password, and logs it. Members of the `sudo` group get full access. Fine-grained rules go in drop-in files under **`/etc/sudoers.d/`**, always edited with **`visudo`**, which checks syntax before saving. Ubuntu 25.10+ uses **sudo-rs**, a Rust rewrite with the same commands.

NOTES

- Password is cached ~15 minutes per terminal
- `sudo -i` = root login shell; `sudo -s` = root shell
- Every sudo use is logged (`journalctl -t sudo`)
- Classic sudo is still available as `sudo.ws`

⚠ DANGER CHECK

- A syntax error in sudoers can lock everyone out of sudo
- Always use `visudo`; keep a second root shell open while editing
- `NOPASSWD: ALL` turns any compromise of that user into root

USING AND CONFIGURING SUDO

```
sudo apt update                          # one command as root
sudo -i                                  # root shell (exit to leave)
sudo -u postgres psql                    # run as another user
sudo -l                                  # what am I allowed to do?

sudo visudo -f /etc/sudoers.d/deploy  # create a drop-in safely
# deploy may restart one service without a password:
#   deploy ALL=(root) NOPASSWD: /usr/bin/systemctl restart myapp
sudo visudo -c                           # check all sudoers files
sudo chmod 440 /etc/sudoers.d/deploy

journalctl -t sudo --since today      # audit sudo use
```

CODE EXPLAINED

- → `sudo -u user` — run as someone other than root
- → `sudo -l` — list your allowed commands
- → `visudo -f` — edit a drop-in with syntax check
- → rule format — `who where=(as_whom) what`
- → `visudo -c` — validate before logging out

OTHER DISTROS

- Debian — sudo optional; root password may be set
- Fedora/RHEL — `wheel` group instead of `sudo`
- macOS — admin users can sudo; same sudoers syntax

PREFER / AVOID

- Avoid `sudo su -` · Prefer `sudo -i`
- Avoid editing `/etc/sudoers` directly · Prefer `/etc/sudoers.d/` + `visudo`

SUDO OPTIONS

OPTION	DOES
`-i`	login shell as root
`-u user`	run as user
`-l`	list rights
`-k`	forget cached password
`-E`	keep environment
`-e (sudoedit)`	edit file safely

SUDOERS RULES

RULE	MEANING
`%sudo ALL=(ALL:ALL) ALL`	sudo group: everything
`bob ALL=(ALL) ALL`	bob: everything
`deploy ALL=(root) NOPASSWD: CMD`	one command, no password
`Defaults timestamp_timeout=5`	cache 5 minutes

⚠ COMMON MISTAKES

- `sudo echo x > /etc/file` fails: the redirect runs as you; use `| sudo tee`
- Drop-in filenames with `.` or `~` are ignored by sudo
- Granting `NOPASSWD` on editors or shells: instant root escape

✓ TIPS & BEST PRACTICES

- Keep a root shell open while changing sudoers or SSH
- Use `echo text | sudo tee -a /etc/file` to append as root
- Prefer narrow, full-path command rules for automation

QUICK SUMMARY

- `sudo` — one command as root, logged
- `sudo -i` — root shell when needed
- `/etc/sudoers.d/` + `visudo` — safe rules
- sudo-rs — default from 25.10

umask, ACLs & Special Bits

DESCRIPTION

Beyond basic rwx: **umask** sets default permissions for new files; **ACLs** grant access to extra users or groups without changing ownership; **setgid** directories make new files inherit the folder's group (great for shared folders); the **sticky bit** stops users deleting each other's files; **chattr +i** makes a file immutable even to root.

NOTES

- Default umask `0002` (user) → files 664, dirs 775
- ACLs show as `+` at the end of `ls -l` modes
- `/tmp` uses the sticky bit (`drwxrwxrwt`)
- `passwd` uses setuid to edit `/etc/shadow`

SHARED FOLDER DONE RIGHT

```
umask                           # 0002
umask 027                       # new files 640, dirs 750 (this shell)

sudo mkdir /srv/shared
sudo chgrp devs /srv/shared
sudo chmod 2775 /srv/shared     # setgid: new files get group devs
ls -ld /srv/shared              # drwxrwsr-x ... devs

sudo setfacl -m u:alice:rwx /srv/reports     # extra user
sudo setfacl -d -m g:devs:rx /srv/reports    # default for new files
getfacl /srv/reports

sudo chmod 1777 /srv/dropbox    # sticky: delete only your own
sudo chattr +i /etc/resolv.conf # immutable
lsattr /etc/resolv.conf         # ----i---------
sudo chattr -i /etc/resolv.conf
```

CODE EXPLAINED

- → `umask 027` — remove w for group, all for others
- → `2775` — leading 2 = setgid
- → `setfacl -m u:alice:rwx` — add a user ACL entry
- → `-d` — default ACL inherited by new files
- → `1777` — leading 1 = sticky bit

OTHER DISTROS

- Debian/Fedora — same tools; Fedora also has SELinux contexts
- macOS — ACLs via `chmod +a` , `chflags uchg` for immutable

PREFER / AVOID

- Avoid `chmod 777` for shared dirs · Prefer setgid + group + umask
- Avoid setuid scripts · Prefer a sudoers rule

SPECIAL BITS

BIT	OCTAL	EFFECT
setuid	4xxx	runs as file owner
setgid (file)	2xxx	runs as file group
setgid (dir)	2xxx	new files inherit group
sticky	1xxx	only owner can delete

UMASK RESULTS

UMASK	FILES / DIRS
0022	644 / 755
0002	664 / 775
0027	640 / 750
0077	600 / 700

⚠ COMMON MISTAKES

- Setting setgid but leaving umask 022: group members still can't write
- Forgetting default ACLs: new files don't get the extra access
- Leaving `chattr +i` on and wondering why updates fail

✓ TIPS & BEST PRACTICES

- Find setuid files when auditing: `find / -perm -4000 -type f 2>/dev/null`
- Set umask for a service with `UMask=` in its unit file
- Use ACLs sparingly; groups are easier to reason about

QUICK SUMMARY

- `umask` — default permissions
- setgid dir — shared group folders
- ACLs — extra users/groups
- sticky / immutable — protect files

apt Basics

DESCRIPTION

apt installs and updates software from Ubuntu's repositories, resolving dependencies for you. Always run **apt update** first to refresh the package lists. Ubuntu 25.04+ ships **apt 3**, with clearer colored output and a solver that explains conflicts; the commands are the same.

NOTES

- `apt` = interactive; `apt-get` = stable output for scripts
- `remove` keeps config files; `purge` deletes them
- Package lists live in `/var/lib/apt/lists/`
- Downloaded `.deb` s cache in `/var/cache/apt/archives/`

EVERYDAY APT

```
sudo apt update                          # refresh lists
apt list --upgradable                    # what would change
sudo apt upgrade                         # install updates
sudo apt install nginx htop              # install (+ deps)
sudo apt install --no-install-recommends curl
apt search "image viewer"                # find packages
apt show nginx                           # version, deps, description
apt policy nginx                         # installed vs candidate, source
sudo apt remove nginx                    # keep /etc/nginx
sudo apt purge nginx                     # remove config too
sudo apt autoremove --purge              # unused dependencies
sudo apt clean                           # empty the download cache
apt list --installed | wc -l
```

CODE EXPLAINED

- → `update` — refresh, doesn't install anything
- → `upgrade` — newer versions of installed packages
- → `--no-install-recommends` — skip optional extras
- → `apt policy` — which repo a version comes from
- → `autoremove` — drop orphaned dependencies

OTHER DISTROS

- Debian — identical `apt`
- Fedora/RHEL — `dnf install/remove/search/info`
- macOS — `brew install/uninstall/search/info`

PREFER / AVOID

- Avoid `apt` in shell scripts · Prefer `apt-get -y` (stable output)
- Avoid downloading random `.deb` s · Prefer repos and PPAs you trust

APT VS APT-GET

TASK	APT	APT-GET / APT-CACHE
`refresh`	apt update	apt-get update
`install`	apt install	apt-get install
`search`	apt search	apt-cache search
`info`	apt show	apt-cache show
`upgrade all`	apt full-upgrade	apt-get dist-upgrade

UPGRADE FLAVORS

COMMAND	DOES
`apt upgrade`	never removes packages
`apt full-upgrade`	may remove to resolve
`unattended-upgrade`	security only, automatic (18)
`do-release-upgrade`	new Ubuntu version (18)

⚠ COMMON MISTAKES

- Installing without `apt update` on a new box: 404 errors for old package versions
- Answering "Y" without reading what `full-upgrade` will remove
- Running two apt commands at once: "Could not get lock"

✓ TIPS & BEST PRACTICES

- `apt-mark showmanual` lists what you installed on purpose
- `apt changelog <pkg>` shows what changed
- Wait for `unattended-upgrades` to finish instead of killing its lock

QUICK SUMMARY

- `apt update` → `apt upgrade` — keep current
- `install` / `remove` / `purge` — manage packages
- `search` / `show` / `policy` — investigate
- `apt-get` — for scripts

Repositories & PPAs

DESCRIPTION

apt downloads from **repositories** listed in source files. Ubuntu's archive has four **components**: `main` and `restricted` (supported by Canonical), `universe` and `multiverse` (community, non-free). Since 24.04, sources use the **deb822** format in `/etc/apt/sources.list.d/ubuntu.sources`. Add third-party software with a **PPA** or a vendor repo pinned to its own signing key.

NOTES

- Old one-line `sources.list` format still works
- Each repo should be tied to one key via `Signed-By`
- `apt-key` is deprecated and removed in newer releases
- PPAs are built by individuals; trust carefully

/ETC/APT/SOURCES.LIST.D/UBUNTU.SOURCES

```
Types: deb
URIs: http://archive.ubuntu.com/ubuntu/
Suites: resolute resolute-updates resolute-backports
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

CODE EXPLAINED

- → `Suites` — release + `-updates` / `-backports`
- → `Components` — which parts of the archive
- → `Signed-By` — only this key may sign this repo
- → `add-apt-repository ppa:` — adds source + key
- → `$VERSION_CODENAME` — matches your release

ADD A PPA AND A VENDOR REPO

```
sudo add-apt-repository ppa:deadsnakes/ppa      # extra Pythons
sudo add-apt-repository --remove ppa:deadsnakes/ppa

# vendor repo with its own key (example: Docker)
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
     -o /etc/apt/keyrings/docker.asc
. /etc/os-release
echo "deb [signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $VERSION_CODENAME stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
```

OTHER DISTROS

- Debian — `debian.sources`, components main/contrib/non-free
- Fedora — `.repo` files in `/etc/yum.repos.d/`, COPR ≈ PPA
- macOS — `brew tap user/repo`

PREFER / AVOID

- Avoid `apt-key add` · Prefer a keyring file + `signed-by`
- Avoid piping install scripts to bash · Prefer a repo you can inspect

COMPONENTS

COMPONENT	CONTENTS
`main`	free, Canonical-supported
`restricted`	non-free drivers, supported
`universe`	free, community
`multiverse`	non-free, community

SUITES

SUITE	CONTENTS
`resolute`	release as shipped
`resolute-updates`	bug fixes
`resolute-security`	security fixes
`resolute-backports`	newer versions
`resolute-proposed`	testing (avoid)

⚠ COMMON MISTAKES

- Adding a repo for another release codename: broken dependencies
- Leaving dead PPAs that break `apt update` after upgrades
- Enabling `-proposed` on a production machine

✓ TIPS & BEST PRACTICES

- Run `ppa-purge ppa:x/y` to downgrade back to Ubuntu's versions
- Prefer official vendor repos for Docker, Node, PostgreSQL
- List all sources: `grep -r ^ /etc/apt/sources.list.d/`

QUICK SUMMARY

- components — main, restricted, universe, multiverse
- deb822 `.sources` — 24.04+ format
- PPA — `add-apt-repository ppa:...`
- `signed-by` — one key per repo

dpkg & .deb Files

DESCRIPTION

dpkg is the low-level tool that actually installs `.deb` packages; `apt` sits on top and handles downloads and dependencies. Use `dpkg` to **query** what's installed and which package owns a file. To install a downloaded `.deb`, prefer `apt install ./file.deb` so dependencies are resolved.

NOTES

- `dpkg -i` doesn't fetch dependencies
- `./` is required so apt treats it as a file
- `apt-mark hold` freezes a package version
- Interrupted installs: `sudo dpkg --configure -a`

⚠ DANGER CHECK

- Forcing installs (`--force-all`) can leave a broken package database
- Holding security-critical packages leaves them unpatched
- Removing packages marked essential (libc, apt) breaks the system

INSTALL, QUERY, REPAIR

```
sudo apt install ./code_1.95_amd64.deb    # resolves deps
sudo dpkg -i some-tool.deb                # low-level, no deps
sudo apt --fix-broken install             # pull missing deps

dpkg -l | grep nginx                      # installed? (ii = yes)
dpkg -L nginx-common | head               # files from a package
dpkg -S /usr/sbin/nginx                   # which package owns it
dpkg-deb -c tool.deb                      # list a .deb's contents
dpkg -s nginx | grep Version

sudo apt-mark hold linux-image-generic    # freeze version
apt-mark showhold
sudo apt-mark unhold linux-image-generic

sudo dpkg --configure -a                  # finish interrupted setup
```

CODE EXPLAINED

- → `apt install ./x.deb` — the right way to install files
- → `--fix-broken install` — repair after `dpkg -i`
- → `dpkg -l` — `ii` means installed fine
- → `dpkg -S path` — reverse lookup
- → `apt-mark hold` — skip in upgrades

OTHER DISTROS

- Debian — identical
- Fedora — `rpm -qa`, `rpm -ql`, `rpm -qf`, `dnf versionlock`
- macOS — `.pkg` with `installer -pkg`; `pkgutil --files`

PREFER / AVOID

- Avoid `dpkg -i` then guessing deps · Prefer `apt install ./file.deb`
- Avoid permanent holds · Prefer documented, temporary holds

DPKG -L STATUS

CODE	MEANING
ii	installed OK
rc	removed, config remains
iU	unpacked, not configured
iF	half-configured (failed)
hi	on hold, installed

QUERY CHEATS

COMMAND	ANSWERS
`dpkg -L pkg`	what files?
`dpkg -S file`	which package?
`apt-cache rdepends pkg`	who needs it?
`apt-cache depends pkg`	what it needs
`dpkg -l 'linux-image*'`	installed kernels

⚠ COMMON MISTAKES

- Leaving `rc` packages: clear with `sudo apt purge ~c` (apt patterns)
- Killing apt mid-install, then ignoring `dpkg --configure -a`
- Holding a package and forgetting it for years

✓ TIPS & BEST PRACTICES

- `apt purge '~c'` removes all leftover configs
- Check `.deb` contents with `dpkg-deb -c` before installing
- `debsums` verifies installed files against package checksums

QUICK SUMMARY

- `apt install ./file.deb` — install with deps
- `dpkg -l / -L / -S` — query
- `apt-mark hold` — freeze versions
- `dpkg --configure -a` — repair

Snap, Flatpak & Other Formats

DESCRIPTION

Snaps are Canonical's self-contained packages: they bundle their dependencies, update automatically, and run confined. Ubuntu ships some apps (Firefox, LXD) as snaps by default. **Flatpak** is a similar format popular on other desktops, with apps from Flathub. **AppImage** is a single executable file with no install step.

NOTES

- Snaps auto-refresh several times a day by default
- `--classic` snaps run unconfined (editors, SDKs)
- Some apt packages are transitional wrappers that install the snap
- Flatpak is not installed by default on Ubuntu

SNAP & FLATPAK BASICS

```
snap find "markdown editor"
sudo snap install code --classic        # VS Code, unconfined
sudo snap install lxd --channel=6/stable
snap list                               # installed + versions
snap info lxd                           # channels, publisher
sudo snap refresh                       # update all now
sudo snap refresh --hold=72h            # pause auto-updates
sudo snap revert lxd                    # back to previous revision
sudo snap remove --purge lxd

sudo apt install flatpak
flatpak remote-add --if-not-exists flathub \
    https://dl.flathub.org/repo/flathub.flatpakrepo
flatpak install flathub org.gimp.GIMP
flatpak update
```

CODE EXPLAINED

- → `--classic` — needs full system access
- → `--channel=6/stable` — track + risk level
- → `snap revert` — instant rollback
- → `--hold` — delay auto-refresh
- → Flathub — main Flatpak app store

OTHER DISTROS

- Debian — apt first; Snap and Flatpak both optional
- Fedora — Flatpak + Flathub built in, no snaps by default
- macOS — Homebrew casks, Mac App Store

PREFER / AVOID

- Avoid mixing apt and snap versions of the same app · Prefer one source
- Avoid disabling snap refresh forever · Prefer `--hold` for a set time

SNAP CHANNELS

PART	EXAMPLE
track	6, latest, 1.30
risk	stable / candidate / beta / edge
full	6/stable
switch	snap refresh --channel=6/edge lxd

WHICH FORMAT?

NEED	USE
server packages	apt
latest desktop app	snap or flatpak
CLI dev tools	snap --classic or apt
portable single file	AppImage
GNOME apps on any distro	Flatpak

⚠ COMMON MISTAKES

- Snap apps can't see files outside `$HOME` by default (confinement)
- Old revisions eat disk: snap keeps 2–3 per app
- Running AppImages that need `libfuse2` without installing it

✓ TIPS & BEST PRACTICES

- Limit kept revisions: `sudo snap set system refresh.retain=2`
- Schedule refreshes: `sudo snap set system refresh.timer=sat,03:00`
- Check snap permissions with `snap connections <app>`

QUICK SUMMARY

- `snap install` — confined, auto-updating apps
- channels — track/risk
- Flatpak + Flathub — alternative app store
- apt for servers, snaps/flatpaks for desktop apps

Updates & Release Upgrades

DESCRIPTION

unattended-upgrades installs security updates automatically every day; it is enabled by default on Ubuntu. **needrestart** tells you which services need restarting after library updates. Moving to a new Ubuntu version is a separate, bigger step: **do-release-upgrade**, normally from one LTS to the next once the first point release is out.

NOTES

- LTS → LTS upgrades are offered after the `.1` point release
- Back up and read release notes before upgrading
- Run release upgrades in `tmux` / `screen` over SSH
- Kernel updates need a reboot (or Livepatch, sheet 36)

⚠ DANGER CHECK

- A release upgrade can break PPAs, custom configs and third-party repos
- Losing SSH mid-upgrade can leave a half-upgraded system
- Snapshot the VM or back up before `do-release-upgrade`

AUTOMATIC UPDATES & RELEASE UPGRADE

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades    # enable
cat /etc/apt/apt.conf.d/20auto-upgrades
# APT::Periodic::Update-Package-Lists "1";
# APT::Periodic::Unattended-Upgrade "1";
sudo unattended-upgrade --dry-run --debug          # test
less /var/log/unattended-upgrades/unattended-upgrades.log

cat /var/run/reboot-required.pkgs 2>/dev/null      # why reboot?
sudo needrestart -r l                              # list restarts

# new Ubuntu version (LTS → LTS)
sudo apt update && sudo apt full-upgrade
sudo do-release-upgrade                            # follow prompts
sudo do-release-upgrade -c                         # just check
```

CODE EXPLAINED

- → `dpkg-reconfigure -plow` — turn auto updates on
- → `20auto-upgrades` — daily update + upgrade
- → `--dry-run --debug` — see what would happen
- → `reboot-required.pkgs` — packages needing reboot
- → `do-release-upgrade` — move to next release

OTHER DISTROS

- Debian — `unattended-upgrades` too; release upgrade by editing sources
- Fedora — `dnf-automatic`; `dnf system-upgrade`
- macOS — automatic updates in System Settings

PREFER / AVOID

- Avoid skipping security updates · Prefer unattended-upgrades + regular reboots
- Avoid upgrading on day one of a release · Prefer waiting for `.1`

50UNATTENDED-UPGRADES OPTIONS

OPTION	EFFECT
`Allowed-Origins`	which suites auto-update
`Automatic-Reboot "true"`	reboot when needed
`Automatic-Reboot-Time "03:30"`	when to reboot
`Remove-Unused-Dependencies`	autoremove
`Mail "me@x.com"`	email reports

RELEASE UPGRADE PROMPTS

SETTING	IN /ETC/UPDATE-MANAGER/RELEASE-UPGRADES
`Prompt=lts`	offer LTS only (default on LTS)
`Prompt=normal`	offer every release
`Prompt=never`	don't offer

⚠ COMMON MISTAKES

- Killing `unattended-upgr` because it holds the apt lock
- Never rebooting: patched kernels and libraries aren't in use yet
- Jumping two LTS versions at once: upgrade one LTS at a time

✓ TIPS & BEST PRACTICES

- Enable `Automatic-Reboot` on servers that can take a 1-minute blip
- Disable third-party repos before a release upgrade, re-add after
- Check `/var/log/dist-upgrade/` if an upgrade fails

QUICK SUMMARY

- `unattended-upgrades` — daily security fixes
- `needrestart` / `reboot-required` — apply them
- `do-release-upgrade` — LTS → LTS
- back up first; use tmux over SSH

Processes

DESCRIPTION

A **process** is a running program with a **PID**, an owner, and resource usage. **ps** takes a snapshot, **top**/**htop** show live usage, and **kill** sends **signals**: TERM asks a process to exit cleanly, KILL forces it. Shell **job control** pauses, backgrounds and resumes commands you started.

NOTES

- `kill` sends `TERM` (15) by default
- `KILL` (9) can't be caught: no cleanup happens
- Niceness -20 (high priority) to 19 (low)
- Zombie (`Z`) = finished, parent hasn't collected it

INSPECT AND CONTROL

```
ps aux --sort=-%mem | head -5        # top memory users
ps -ef --forest | less               # tree of parent/child
pgrep -a nginx                       # PIDs + command lines
pidof sshd
htop                                 # F6 sort, F9 kill, F10 quit
kill 4321                            # polite: SIGTERM
kill -9 4321                         # forced: SIGKILL (last resort)
pkill -f "python worker.py"          # by command line
sudo kill -HUP $(pidof nginx | cut -d' ' -f1)   # reload config
nice -n 10 tar -czf big.tgz /data    # start with low priority
sudo renice -n 5 -p 4321             # change priority
sleep 300 &                          # background job
jobs; fg %1                          # list, bring to front
nohup ./long-task.sh > task.log 2>&1 &   # survive logout
```

CODE EXPLAINED

- → `ps aux` — all processes, user-oriented
- → `pgrep -a` — find by name with full command
- → `kill` then `kill -9` — escalate only if needed
- → `-HUP` — many daemons reload config
- → `nohup ... &` — keep running after logout

OTHER DISTROS

- Debian/Fedora — same procps tools
- macOS — `ps aux` works; `top -o mem` ; no `htop` by default

PREFER / AVOID

- Avoid `kill -9` first · Prefer `TERM` , wait, then `KILL`
- Avoid `nohup` for real services · Prefer a systemd unit (sheet 21)

SIGNALS

SIGNAL	#	MEANING
SIGTERM	15	please exit (default)
SIGKILL	9	kill now, no cleanup
SIGHUP	1	hang up / reload
SIGINT	2	Ctrl+C
SIGSTOP / SIGCONT	19 / 18	pause / resume

PS STAT CODES

CODE	MEANING
R	running
S	sleeping
D	uninterruptible (I/O)
Z	zombie
T	stopped
+ / s	foreground / session leader

⚠ COMMON MISTAKES

- `kill -9` on a database: risk of corruption and lost writes
- `killall` on another Unix kills *all* processes; on Linux it matches names
- Many `D` state processes usually mean a disk or NFS problem

✓ TIPS & BEST PRACTICES

- Use `btop` for a prettier dashboard (apt install btop)
- Use `tmux` for long interactive jobs over SSH
- `lsof -p PID` shows files and sockets a process holds

QUICK SUMMARY

- `ps` / `htop` — see processes
- `kill` — send signals (TERM first)
- `nice` / `renice` — priority
- `&` , `jobs` , `fg` — job control

systemd & systemctl

DESCRIPTION

systemd starts the system and manages long-running services. Each thing it manages is a **unit** (`.service`, `.timer`, `.socket`, `.mount`). **systemctl** controls units: start and stop them now, **enable** them to start at boot, and check **status** with the latest log lines.

NOTES

- `enable` = start at boot; `start` = start now
- `enable --now` does both
- `reload` re-reads config without dropping connections (if supported)
- `--user` manages your own per-user services

MANAGING SERVICES

```
systemctl status nginx                  # state + recent logs
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx
sudo systemctl reload nginx             # graceful config reload
sudo systemctl enable --now ssh         # boot + now
sudo systemctl disable --now apache2
systemctl is-active nginx               # active / inactive
systemctl is-enabled nginx              # enabled / disabled
systemctl list-units --type=service --state=running
systemctl --failed                      # anything broken?
systemctl cat nginx                     # show unit file(s)
sudo systemctl mask cups                # block from starting at all
systemctl list-dependencies multi-user.target | head
```

CODE EXPLAINED

- → `status` — running? since when? last logs
- → `reload` vs `restart` — keep vs drop connections
- → `enable --now` — common one-liner
- → `--failed` — quick health check
- → `mask` — stronger than disable

OTHER DISTROS

- Debian/Fedora — same systemd and `systemctl`
- macOS — `launchctl` + plist files; `brew services start`

PREFER / AVOID

- Avoid old `service nginx restart` · Prefer `systemctl restart nginx`
- Avoid restarting to apply config · Prefer `reload` when supported

UNIT TYPES

TYPE	MANAGES
`.service`	daemon / program
`.timer`	scheduled jobs (23)
`.socket`	socket activation
`.mount`	filesystem mounts
`.target`	group of units (like runlevels)
`.path`	watch files

TARGETS

TARGET	MEANING
`multi-user.target`	normal server boot
`graphical.target`	desktop
`rescue.target`	single-user repair
`emergency.target`	minimal shell
`default.target`	symlink to boot target

⚠ COMMON MISTAKES

- `enable` without `start` and wondering why it's not running
- Restarting SSH with a broken config: test with `sshd -t` first
- Editing unit files in `/usr/lib/systemd` (overwritten by updates)

✓ TIPS & BEST PRACTICES

- `systemctl status` + `journalctl -u name -e` solve most issues
- Override units with `systemctl edit` (sheet 21)
- `systemctl list-timers` shows scheduled jobs

QUICK SUMMARY

- units — services, timers, mounts, targets
- `start/stop/restart/reload` — now
- `enable/disable` — at boot
- `status` / `--failed` — health

Writing Service Units

DESCRIPTION

Run your own app as a proper service by writing a **unit file** in `/etc/systemd/system/`. systemd then starts it at boot, restarts it if it crashes, runs it as an unprivileged user, and sends its output to the journal. To tweak a packaged service, use **`systemctl edit`** to create an override instead of editing the vendor file.

NOTES

- Run `daemon-reload` after creating or editing units
- Use absolute paths in `ExecStart`
- stdout/stderr go to the journal automatically
- Overrides land in `/etc/systemd/system/<unit>.d/override.conf`

/ETC/SYSTEMD/SYSTEM/TODO-API.SERVICE

```
[Unit]
Description=Todo API (FastAPI)
After=network-online.target postgresql.service
Wants=network-online.target

[Service]
User=todo
Group=todo
WorkingDirectory=/srv/todo-api
EnvironmentFile=/etc/todo-api.env
ExecStart=/srv/todo-api/.venv/bin/fastapi run app/main.py --port 8000
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/srv/todo-api/data

[Install]
WantedBy=multi-user.target
```

CODE EXPLAINED

- → `After=` / `Wants=` — start order and soft dependency
- → `User=` — never run apps as root
- → `EnvironmentFile=` — secrets outside the unit
- → `Restart=on-failure` — auto-restart crashes
- → `ProtectSystem=strict` — read-only OS for the service

OTHER DISTROS

- Debian/Fedora — identical unit files
- macOS — launchd plist in `/Library/LaunchDaemons/`

LOAD IT

```
sudo systemctl daemon-reload
sudo systemctl enable --now todo-api
sudo systemctl edit nginx     # opens an override.conf
```

PREFER / AVOID

- Avoid `nohup` / `screen` for services · Prefer a unit file
- Avoid copying vendor units to edit · Prefer `systemctl edit` overrides

[SERVICE] KEYS

KEY	PURPOSE
`Type=simple / notify / oneshot`	how startup is detected
`ExecStart / ExecReload`	commands
`Restart= / RestartSec=`	crash policy
`Environment= / EnvironmentFile=`	variables
`User= / Group=`	run as
`MemoryMax= / CPUQuota=`	resource limits

HARDENING KEYS

KEY	EFFECT
`NoNewPrivileges=true`	no setuid escalation
`ProtectSystem=strict`	read-only /usr, /etc
`ProtectHome=true`	hide /home
`PrivateTmp=true`	own /tmp
`ReadWritePaths=`	allowed writes

⚠ COMMON MISTAKES

- Forgetting `daemon-reload`: systemd keeps the old definition
- Relative paths in `ExecStart`: "Executable path is not absolute"
- Secrets in `Environment=` lines, readable by `systemctl show`

✓ TIPS & BEST PRACTICES

- Check hardening with `systemd-analyze security todo-api`
- Use `Type=notify` for apps that support sd_notify
- Test the exact `ExecStart` command as that user first

QUICK SUMMARY

- unit file — `[Unit]` `[Service]` `[Install]`
- `User=` + `Restart=on-failure` — safe, resilient
- `daemon-reload` → `enable --now`
- `systemctl edit` — override packaged units

Logs & journalctl

DESCRIPTION

systemd's **journal** collects logs from the kernel, services and anything they print. **`journalctl`** filters it by unit, time, priority and boot. Some programs still write text files under **`/var/log`** (nginx, apt, auth), which **logrotate** compresses and prunes. `dmesg` shows kernel messages like disk and USB events.

NOTES

- Journal is persistent on Ubuntu (`/var/log/journal`)
- Users in `adm` or `systemd-journal` can read all logs
- `-e` jumps to the end; `-r` newest first
- `--vacuum-size=` limits journal disk use

FIND THE RIGHT LOG LINES

```
journalctl -u nginx -e                    # one service, jump to end
journalctl -u todo-api -f                 # follow live
journalctl --since "1 hour ago" -p err    # errors in the last hour
journalctl -b -1 -p warning               # warnings from previous boot
journalctl _PID=4321                      # one process
journalctl -k                             # kernel messages (= dmesg)
journalctl -u ssh --since today -o short-iso | grep Failed
journalctl --disk-usage
sudo journalctl --vacuum-time=14d         # keep 14 days
sudo dmesg -T | tail                      # kernel, readable times
tail -F /var/log/nginx/access.log         # classic text logs
sudo less /var/log/auth.log               # logins, sudo
cat /etc/logrotate.d/nginx                # rotation rules
```

CODE EXPLAINED

- → `-u unit` — filter by service
- → `--since` / `--until` — natural time phrases work
- → `-p err` — priority err and worse
- → `-b -1` — previous boot (crash debugging)
- → `--vacuum-time` — prune old journal files

OTHER DISTROS

- Debian/Fedora — same journalctl; Fedora has no `/var/log/syslog`
- macOS — `log show --last 1h`, `log stream`

PREFER / AVOID

- Avoid `grep` through all of `/var/log` · Prefer `journalctl -u` filters
- Avoid unbounded logs · Prefer `SystemMaxUse=` in journald.conf

PRIORITIES

LEVEL	NAME
0–2	emerg, alert, crit
3	err
4	warning
5–6	notice, info
7	debug

LOG FILES

FILE	CONTAINS
/var/log/syslog	general system log
/var/log/auth.log	logins, sudo, ssh
/var/log/kern.log	kernel
/var/log/apt/history.log	package changes
/var/log/nginx/	web server
/var/log/dpkg.log	dpkg actions

⚠ COMMON MISTAKES

- Looking at the current boot's log when the crash was in the previous one (`-b -1`)
- Letting the journal grow to gigabytes on small disks
- Missing app errors because the app logs to its own file, not stdout

✓ TIPS & BEST PRACTICES

- Set `SystemMaxUse=500M` in `/etc/systemd/journald.conf`
- Log to stdout from your apps; systemd captures it
- `journalctl -o json-pretty` for structured fields

QUICK SUMMARY

- `journalctl -u name` — service logs
- `-f` / `--since` / `-p` / `-b` — filter
- `/var/log` — text logs, rotated by logrotate
- `--vacuum-*` — control disk use

Scheduling Jobs

DESCRIPTION

Run commands on a schedule with **cron** (simple, classic) or **systemd timers** (logged in the journal, can catch up missed runs, dependency-aware). Use **at** for a one-off job later. Cron uses a five-field schedule; timers use `OnCalendar=` expressions like `Mon..Fri 02:30`.

NOTES

- Cron runs with a minimal `PATH` : use full paths
- Cron output is mailed or lost; redirect it to a log
- Timers need a matching `.service` unit
- `Persistent=true` runs missed jobs after boot

CRON

```
crontab -e                          # your user's crontab
# m  h  dom mon dow  command
30   2  *   *   *    /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
*/15 *  *   *   1-5  /usr/bin/curl -fsS https://hc-ping.com/abc
crontab -l                          # list
echo "uptime" | at 17:00            # one-off job
```

CODE EXPLAINED

- → five fields — minute hour day month weekday
- → `*/15` — every 15; `1-5` — Mon–Fri
- → `>> log 2>&1` — keep output and errors
- → `OnCalendar=` — when the timer fires
- → `Persistent=true` — catch up after downtime

SYSTEMD TIMER: BACKUP.TIMER (+ BACKUP.SERVICE)

```
[Unit]
Description=Nightly backup

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true
RandomizedDelaySec=10m

[Install]
WantedBy=timers.target
# sudo systemctl enable --now backup.timer
# systemctl list-timers
```

OTHER DISTROS

- Debian — same cron + timers; Fedora uses `cronie`
- macOS — launchd `StartCalendarInterval` ; cron still works

PREFER / AVOID

- Avoid silent cron jobs · Prefer logging + a health-check ping
- Avoid cron for jobs needing ordering/retries · Prefer systemd timers

CRON EXAMPLES

SCHEDULE	MEANS
`0 * * * *`	every hour
`*/5 * * * *`	every 5 minutes
`0 3 * * 0`	Sundays 03:00
`0 9 1 * *`	1st of month 09:00
`@reboot`	at boot
`@daily`	midnight

ONCALENDAR EXAMPLES

EXPRESSION	MEANS
`hourly / daily / weekly`	shortcuts
`Mon..Fri 08:00`	weekdays 8am
`*-*-01 00:00`	monthly
`*:0/15`	every 15 min
`systemd-analyze calendar '...'`	test an expression

⚠ COMMON MISTAKES

- Commands work in your shell but fail in cron: missing PATH or env vars
- `%` in crontab lines must be escaped as `\%`
- Editing `/etc/crontab` without the user field

✓ TIPS & BEST PRACTICES

- Test schedules at crontab.guru or with `systemd-analyze calendar`
- Use `flock -n /tmp/job.lock cmd` to prevent overlapping runs
- Ping a monitor (Healthchecks.io) at the end of important jobs

QUICK SUMMARY

- cron — `m h dom mon dow cmd`
- systemd timers — logged, persistent
- `at` — one-off jobs
- full paths + logs — reliable jobs

Disks & Partitions

DESCRIPTION

Before you can store data on a new disk you identify it, **partition** it, and create a filesystem on it (sheet 25). **`lsblk`** shows disks and partitions as a tree; **`parted`** or **`fdisk`** edits partition tables. Use **GPT** for modern systems. Device names depend on the bus: `sda` for SATA/USB, `nvme0n1` for NVMe.

NOTES

- NVMe partitions add `p`: `nvme0n1p1`
- Names can change between boots; mount by UUID
- GPT: up to 128 partitions, > 2 TB disks
- GUI option: GNOME Disks or GParted

⚠ DANGER CHECK

- Partitioning the wrong disk destroys its data instantly
- `wipefs -a` and `dd` to a device have no confirmation
- Confirm the disk by size and model with `lsblk -o +MODEL,SERIAL` first

PARTITION A NEW DISK

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,MODEL
# sdb   4T disk               WDC WD40EFRX
sudo fdisk -l /dev/sdb                # details + partition table
sudo parted /dev/sdb --script \
     mklabel gpt \
     mkpart data ext4 1MiB 100%       # one partition, whole disk
lsblk /dev/sdb                        # sdb1 now exists
sudo blkid /dev/sdb1                  # UUID (after mkfs)
sudo wipefs /dev/sdb                  # list signatures (no -a = safe)
sudo smartctl -H /dev/sdb             # disk health (smartmontools)
```

CODE EXPLAINED

- → `lsblk -o ... MODEL` — match the physical disk
- → `mklabel gpt` — new GPT partition table
- → `mkpart ... 1MiB 100%` — aligned, full-size partition
- → `blkid` — UUIDs and filesystem types
- → `smartctl -H` — SMART health check

OTHER DISTROS

- Debian/Fedora — same tools; Fedora defaults to Btrfs
- macOS — `diskutil list`, `diskutil partitionDisk`

PREFER / AVOID

- Avoid MBR on new disks · Prefer GPT
- Avoid relying on `/dev/sdX` names · Prefer UUIDs or `/dev/disk/by-id/`

DEVICE NAMES

NAME	DEVICE
`/dev/sda, sdb`	SATA / SAS / USB disks
`/dev/sda1`	first partition
`/dev/nvme0n1`	first NVMe drive
`/dev/nvme0n1p2`	its 2nd partition
`/dev/vda`	virtio VM disk
`/dev/mapper/...`	LVM / LUKS

TOOLS

TOOL	USE
`lsblk`	overview tree
`fdisk / gdisk`	interactive editing
`parted`	scriptable, big disks
`blkid`	UUIDs, types
`smartctl`	health
`gnome-disks / gparted`	GUI

⚠ COMMON MISTAKES

- Formatting `/dev/sdb` (whole disk) when you meant `/dev/sdb1`
- Unaligned partitions (start at 1MiB to be safe)
- Assuming `sdb` is the same disk after adding hardware

✓ TIPS & BEST PRACTICES

- Use `/dev/disk/by-id/` paths in scripts and ZFS
- Run `sudo partprobe` if the kernel doesn't see new partitions
- Label partitions so `lsblk` output is self-explanatory

QUICK SUMMARY

- `lsblk` — see disks
- `parted mklabel gpt` + `mkpart` — partition
- `blkid` — UUIDs
- triple-check the device name

Filesystems & Mounting

DESCRIPTION

A partition becomes usable once it has a **filesystem** (`mkfs`) and is **mounted** on a directory. **`/etc/fstab`** lists mounts to set up at every boot; always use **UUIDs** there and test with `mount -a` before rebooting. **ext4** is Ubuntu's default; XFS, Btrfs and ZFS are alternatives.

NOTES

- Mount point must be an existing (empty) directory
- `findmnt --verify` checks fstab syntax
- `nofail` lets the system boot if the disk is missing
- Desktop auto-mounts USB drives under `/media/$USER`

⚠ DANGER CHECK

- `mkfs` on the wrong device erases it
- A bad fstab line can drop the server into emergency mode at boot
- Always run `sudo mount -a` (and `findmnt --verify`) before rebooting

FORMAT, MOUNT, PERSIST

```
sudo mkfs.ext4 -L data /dev/sdb1     # create filesystem, label "data"
sudo mkdir -p /srv/data
sudo mount /dev/sdb1 /srv/data       # mount now
df -h /srv/data
sudo blkid /dev/sdb1                 # UUID="3f2a...-..." TYPE="ext4"

# /etc/fstab  (one line)
# UUID=3f2a...  /srv/data  ext4  defaults,nofail  0  2

sudo findmnt --verify                # syntax check
sudo umount /srv/data && sudo mount -a   # test fstab
findmnt /srv/data                    # mounted again? good
sudo umount /media/bob/USB           # eject USB safely
sudo lsof +f -- /srv/data            # who blocks umount?
```

CODE EXPLAINED

- → `mkfs.ext4 -L` — format with a label
- → `mount dev dir` — attach now (not persistent)
- → fstab fields — what, where, type, options, dump, fsck order
- → `nofail` — boot even if missing
- → `mount -a` — mount everything in fstab now

OTHER DISTROS

- Debian — same; Fedora defaults to Btrfs with subvolumes
- macOS — `diskutil mount`, APFS; `/etc/fstab` rarely used

PREFER / AVOID

- Avoid `/dev/sdb1` in fstab · Prefer `UUID=` or `LABEL=`
- Avoid rebooting after fstab edits untested · Prefer `mount -a` first

FSTAB OPTIONS

OPTION	EFFECT
`defaults`	rw, suid, dev, exec, auto, nouser, async
`nofail`	don't block boot
`noatime`	fewer writes
`ro`	read-only
`x-systemd.automount`	mount on first access
`uid=, gid=`	owner (FAT/NTFS)

FILESYSTEMS

TYPE	USE
`ext4`	default, reliable
`xfs`	large files, servers
`btrfs`	snapshots, compression
`zfs`	pools, checksums (27)
`vfat / exfat`	USB, cross-OS
`nfs / cifs`	network (39)

⚠ COMMON MISTAKES

- Typo in fstab → emergency mode; fix from recovery (sheet 46)
- Mounting over a non-empty directory hides its files
- Pulling a USB drive without `umount`: data loss

✓ TIPS & BEST PRACTICES

- Label filesystems and use `LABEL=` for readability
- Use `x-systemd.automount` for network shares
- Check filesystem with `sudo fsck -f /dev/sdb1` (unmounted only)

QUICK SUMMARY

- `mkfs` — create filesystem
- `mount` / `umount` — attach / detach
- `/etc/fstab` + UUID — persistent
- `mount -a` — test before reboot

LVM

DESCRIPTION

LVM (Logical Volume Manager) puts a flexible layer between disks and filesystems. **Physical volumes** (disks/partitions) form a **volume group** (a pool), which is carved into **logical volumes** that behave like partitions but can grow, span disks and be snapshotted. Ubuntu Server uses LVM by default and leaves free space in the pool.

NOTES

- Default server VG: `ubuntu-vg`, LV: `ubuntu-lv`
- Growing ext4/XFS online is safe and common
- Shrinking needs unmounting (ext4) and is riskier
- LVM is not RAID or backup by itself

⚠ DANGER CHECK

- Shrinking an LV smaller than its filesystem destroys data
- `lvremove` / `vgremove` delete without a second chance
- Back up before any shrink or reorganization

GROW ROOT & ADD A DISK

```
sudo vgs                                    # VG size and free space
sudo lvs                                    # logical volumes
sudo pvs                                    # physical volumes

# use the free space Ubuntu left unallocated
sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
df -h /                                     # bigger, no reboot

# add a new disk to the pool
sudo pvcreate /dev/sdc
sudo vgextend ubuntu-vg /dev/sdc
sudo lvcreate -n media -L 500G ubuntu-vg    # new LV
sudo mkfs.ext4 /dev/ubuntu-vg/media

# snapshot before a risky change
sudo lvcreate -s -n root-snap -L 10G /dev/ubuntu-vg/ubuntu-lv
sudo lvremove /dev/ubuntu-vg/root-snap      # when done
```

CODE EXPLAINED

- → `vgs` / `lvs` / `pvs` — summaries
- → `lvextend -r` — grow LV **and** filesystem
- → `+100%FREE` — use all free space in the VG
- → `pvcreate` + `vgextend` — add disk to pool
- → `lvcreate -s` — copy-on-write snapshot

OTHER DISTROS

- Debian/Fedora — same LVM2 tools; Fedora prefers Btrfs
- macOS — APFS containers/volumes play a similar role

PREFER / AVOID

- Avoid filling the VG at install · Prefer leaving free space to grow into
- Avoid keeping LVM snapshots for days · Prefer short-lived snapshots

LAYERS

LAYER	EXAMPLE
PV (physical volume)	/dev/sda3, /dev/sdc
VG (volume group)	ubuntu-vg
LV (logical volume)	/dev/ubuntu-vg/ubuntu-lv
filesystem	ext4 on the LV

COMMON COMMANDS

COMMAND	DOES
`lvextend -r -L +20G lv`	grow by 20 GB
`lvreduce -r -L 50G lv`	shrink (risky)
`lvrename vg old new`	rename
`vgreduce vg /dev/sdc`	remove empty PV
`pvmove /dev/sdb`	move data off a PV

⚠ COMMON MISTAKES

- `lvextend` without `-r`, then `df` still shows the old size
- Snapshots filling up and becoming invalid
- Spanning a VG across disks without RAID: one dead disk loses the LV

✓ TIPS & BEST PRACTICES

- `sudo lvextend -r -l +100%FREE` is the classic post-install fix
- Use `pvmove` to migrate off a failing disk online
- Combine with mdadm RAID for redundancy (sheet 27)

QUICK SUMMARY

- PV → VG → LV — flexible storage
- `lvextend -r` — grow online
- `vgextend` — add disks
- snapshots — short-term safety net

ZFS & Software RAID

DESCRIPTION

ZFS combines RAID, volume management and a filesystem, with checksums that detect silent corruption, cheap **snapshots** and compression. Ubuntu supports it out of the box (`zfsutils-linux`). **mdadm** is classic Linux software RAID under any filesystem. Both protect against disk failure, but **RAID is not backup**.

NOTES

- Mirror (RAID 1): 2+ disks, survives 1 failure per mirror
- raidz1/2: survives 1/2 disk failures
- ZFS likes lots of RAM and whole disks
- Scrub monthly to find and repair bad blocks

⚠ DANGER CHECK

- `zpool create` / `mdadm --create` wipe the member disks
- Use `/dev/disk/by-id/` names, or pools break when `sdX` names change
- Deleting a dataset or destroying a pool is permanent

ZFS MIRROR & MDADM RAID 1

```
sudo apt install zfsutils-linux
sudo zpool create -o ashift=12 tank mirror \
    /dev/disk/by-id/ata-WDC_A /dev/disk/by-id/ata-WDC_B
sudo zfs set compression=lz4 tank
sudo zfs create tank/photos                # dataset at /tank/photos
sudo zfs snapshot tank/photos@2026-11-13
zfs list -t snapshot
sudo zfs rollback tank/photos@2026-11-13   # undo changes
zpool status                               # health, errors
sudo zpool scrub tank

sudo apt install mdadm
sudo mdadm --create /dev/md0 --level=1 \
    --raid-devices=2 /dev/sdb1 /dev/sdc1
cat /proc/mdstat                           # sync progress
sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
```

CODE EXPLAINED

- → `mirror` — two-disk ZFS mirror
- → `ashift=12` — 4K sectors (modern disks)
- → `compression=lz4` — free space, often faster
- → `zfs snapshot` / `rollback` — instant restore points
- → `mdadm --create --level=1` — RAID 1 device

OTHER DISTROS

- Debian — ZFS from contrib (DKMS); Fedora — via OpenZFS repo
- macOS — OpenZFS on OS X port; no built-in software RAID UI beyond Disk Utility

PREFER / AVOID

- Avoid RAID as your only copy · Prefer RAID + off-site backup
- Avoid `sdX` names in pools · Prefer `/dev/disk/by-id/`

LAYOUTS

LAYOUT	USABLE	SURVIVES
`mirror (2 disks)`	1 disk	1 failure
`raidz1 (4 disks)`	3 disks	1 failure
`raidz2 (6 disks)`	4 disks	2 failures
`RAID 5 mdadm (3)`	2 disks	1 failure
`RAID 10 (4)`	2 disks	1 per mirror

HEALTH CHECKS

COMMAND	SHOWS
`zpool status -x`	only problems
`zpool list`	capacity
`zfs list -o name,used,avail`	datasets
`cat /proc/mdstat`	mdadm arrays
`mdadm --detail /dev/md0`	array details

⚠ COMMON MISTAKES

- Mixing disk sizes: pool uses the smallest
- Ignoring `DEGRADED` status until the second disk dies
- Filling ZFS above ~80%: performance drops sharply

✓ TIPS & BEST PRACTICES

- Automate snapshots with `zfs-auto-snapshot` or sanoid
- Send snapshots off-box: `zfs send | ssh backup zfs recv`
- Set up email alerts (`zed` or mdadm `MAILADDR`)

QUICK SUMMARY

- ZFS — RAID + volumes + checksummed FS
- mirror / raidz — redundancy
- snapshots + scrubs — safety and integrity
- mdadm — classic software RAID

Network Basics

DESCRIPTION

The **ip** command shows and changes addresses and routes; **ss** shows which programs are listening on which ports. For troubleshooting, work up the stack: is the interface up, is there an IP, is the gateway reachable (`ping`), does DNS resolve, is the port open (`nc`, `curl`)?

NOTES

- `ifconfig` / `netstat` are legacy (`net-tools`)
- Changes made with `ip` are lost on reboot (use Netplan)
- `ss -tulpn` needs sudo to show process names
- `127.0.0.1` = loopback; `0.0.0.0` = all interfaces

TROUBLESHOOT STEP BY STEP

```
ip -br a                          # interfaces + IPs, one line each
ip route                          # default via 192.168.1.1 dev enp3s0
ping -c 3 192.168.1.1             # gateway reachable?
ping -c 3 1.1.1.1                 # internet by IP?
ping -c 3 ubuntu.com              # DNS working?
sudo ss -tulpn                    # listening TCP/UDP ports + process
ss -tn state established          # current connections
curl -I https://example.com       # HTTP headers only
curl -fsS localhost:8000/health   # local service check
nc -zv 192.168.1.20 22            # is port 22 open?
mtr -rwc 10 ubuntu.com            # path + packet loss report
ip neigh                          # ARP table (LAN neighbors)
```

CODE EXPLAINED

- → `ip -br a` — brief address list
- → `ip route` — default gateway
- → ping gateway → IP → name — isolate the layer
- → `ss -tulpn` — what's listening, by whom
- → `nc -zv` — port check without a client

OTHER DISTROS

- Debian/Fedora — same iproute2 tools
- macOS — `ifconfig`, `netstat -rn`, `lsof -i -P`

PREFER / AVOID

- Avoid `ifconfig` / `netstat` · Prefer `ip` / `ss`
- Avoid testing only by name · Prefer ping by IP then by name

SS FLAGS

FLAG	MEANING
`-t / -u`	TCP / UDP
`-l`	listening only
`-p`	show process
`-n`	numeric, no DNS
`state established`	active connections

LEGACY → MODERN

OLD	NEW
`ifconfig`	ip a
`route -n`	ip route
`netstat -tulpn`	ss -tulpn
`arp -a`	ip neigh
`iwconfig`	iw / nmcli

⚠ COMMON MISTAKES

- Service listening on `127.0.0.1` and wondering why other machines can't reach it
- Blaming the network when the firewall blocks the port (sheet 33)
- Setting an IP with `ip addr add` and losing it on reboot

✓ TIPS & BEST PRACTICES

- `curl -v` shows TLS and redirect details
- `sudo tcpdump -i any port 53` to watch DNS traffic
- Check `sudo ufw status` early when ports are unreachable

QUICK SUMMARY

- `ip a` / `ip route` — addresses, gateway
- `ping` / `mtr` — reachability
- `ss -tulpn` — listening ports
- `curl` / `nc` — service checks

Netplan

DESCRIPTION

Ubuntu configures networking with **Netplan**: YAML files in `/etc/netplan/` that describe interfaces, and Netplan writes the config for the **renderer**: `systemd-networkd` on servers, NetworkManager on desktops. Use **`netplan try`**, which rolls back automatically if you lose connectivity, before `netplan apply`.

NOTES

- YAML uses spaces, never tabs
- Files apply in name order; later ones override
- Config files should be `chmod 600`
- Desktop: change settings in GNOME or `nmcli` instead

⚠ DANGER CHECK

- A wrong static IP or gateway cuts off a remote server
- Use `netplan try` (auto-rollback in 120 s), never straight `apply` over SSH
- Keep console access (VM console, IPMI) when changing networking

/ETC/NETPLAN/50-CLOUD-INIT.YAML → STATIC IP

```yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp3s0:
      dhcp4: false
      addresses: [192.168.1.20/24]
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [1.1.1.1, 9.9.9.9]
        search: [home.lan]
```

CODE EXPLAINED

- → `renderer: networkd` — server backend
- → `addresses` — IP/prefix (CIDR)
- → `routes: to: default` — gateway (old `gateway4` is deprecated)
- → `nameservers` — DNS servers + search domains
- → `netplan try` — automatic rollback safety

APPLY SAFELY

```
sudo chmod 600 /etc/netplan/*.yaml
sudo netplan generate          # syntax check
sudo netplan try               # press Enter to keep, else rollback
sudo netplan apply
netplan status                 # current state per interface
```

OTHER DISTROS

- Debian — `/etc/network/interfaces` (ifupdown) or NetworkManager
- Fedora — NetworkManager `nmcli` / keyfiles
- macOS — `networksetup`, System Settings

PREFER / AVOID

- Avoid `gateway4:` · Prefer `routes: - to: default`
- Avoid static IPs set on the server when DHCP reservations work · Prefer router reservations

COMMON KEYS

KEY	EXAMPLE
`dhcp4`	true / false
`addresses`	[10.0.0.5/24]
`routes`	to: default, via: 10.0.0.1
`nameservers.addresses`	[1.1.1.1]
`optional: true`	don't wait at boot
`mtu`	9000

WI-FI & OTHERS

SECTION	USE
`wifis:`	SSID + password
`bonds:`	link aggregation
`bridges:`	VMs / LXD bridge
`vlans:`	tagged VLANs
`nmcli dev wifi connect`	desktop Wi-Fi

⚠ COMMON MISTAKES

- Tabs or bad indentation: "Invalid YAML"
- Two files both configuring the same interface with conflicting settings
- cloud-init rewriting `50-cloud-init.yaml` on boot (disable its network config)

✓ TIPS & BEST PRACTICES

- Disable cloud-init networking: `network: {config: disabled}` in `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg`
- Check interface names with `ip -br link`
- Use `netplan get` to print the merged config

QUICK SUMMARY

- `/etc/netplan/*.yaml` — network config
- static IP — addresses, routes, nameservers
- `netplan try` — safe apply with rollback
- renderer — networkd (server) / NetworkManager (desktop)

DNS & Hostnames

DESCRIPTION

Ubuntu resolves names through **systemd-resolved**, a local DNS stub at `127.0.0.53` that `/etc/resolv.conf` points to. **`resolvectl`** shows which DNS servers each interface uses and lets you flush the cache. **`/etc/hosts`** overrides DNS for specific names, and **Avahi** answers `hostname.local` on your LAN.

NOTES

- Don't edit `/etc/resolv.conf` directly; it's managed
- Set DNS servers in Netplan (sheet 29)
- `dig` is in the `bind9-dnsutils` package
- mDNS (`.local`) needs `avahi-daemon` (desktop default)

NAMES IN, NAMES OUT

```
hostnamectl                              # Static hostname: homeserver
sudo hostnamectl set-hostname homeserver
grep homeserver /etc/hosts               # 127.0.1.1 homeserver
resolvectl status                        # DNS servers per link
resolvectl query ubuntu.com              # resolve via resolved
dig +short ubuntu.com A                  # direct DNS query
dig @1.1.1.1 example.com MX              # ask a specific server
nslookup example.com
sudo resolvectl flush-caches
getent hosts nas.home.lan                # what apps will see
echo "192.168.1.30 nas nas.home.lan" | sudo tee -a /etc/hosts
sudo apt install avahi-daemon            # then: ssh bob@homeserver.local
```

CODE EXPLAINED

- → `hostnamectl set-hostname` — permanent name
- → `127.0.1.1` line — lets the host resolve itself
- → `resolvectl status` — which DNS is really used
- → `dig +short` — just the answer
- → `getent hosts` — uses the full resolver order

OTHER DISTROS

- Debian — resolved optional; often plain `/etc/resolv.conf`
- Fedora — systemd-resolved too
- macOS — `scutil --dns`, `dscacheutil -flushcache`

PREFER / AVOID

- Avoid editing `/etc/resolv.conf` · Prefer Netplan nameservers / `resolvectl`
- Avoid long `/etc/hosts` lists · Prefer local DNS (router, Pi-hole)

RECORD TYPES

TYPE	HOLDS
`A / AAAA`	IPv4 / IPv6 address
`CNAME`	alias to another name
`MX`	mail servers
`TXT`	verification, SPF
`NS`	name servers
`PTR`	reverse lookup

RESOLUTION ORDER

STEP	SOURCE
1	/etc/hosts
2	mDNS (.local) via Avahi
3	systemd-resolved cache
4	upstream DNS servers
`config`	/etc/nsswitch.conf hosts: line

⚠ COMMON MISTAKES

- Hostname changed but not in `/etc/hosts` : "sudo: unable to resolve host"
- Testing with `dig` (bypasses `/etc/hosts`) and expecting hosts entries
- Port 53 conflicts when installing Pi-hole/dnsmasq next to resolved

✓ TIPS & BEST PRACTICES

- Use `dig +trace` to follow a lookup from the root servers
- Run Pi-hole or AdGuard Home for a LAN-wide DNS
- Disable the resolved stub (`DNSStubListener=no`) before running a DNS server

QUICK SUMMARY

- `hostnamectl` — set the name
- systemd-resolved — local DNS stub
- `resolvectl` / `dig` — inspect DNS
- `/etc/hosts` / `.local` — local names

SSH

DESCRIPTION

SSH gives you an encrypted shell on remote machines. Log in with **keys** instead of passwords: generate a key pair, copy the public key to the server, then turn off password logins. A **`~/.ssh/config`** file turns long commands into short aliases. SSH can also **forward ports**, tunneling a remote service to your laptop.

NOTES

- Server package: `openssh-server`; service name `ssh`
- Put server settings in `/etc/ssh/sshd_config.d/*.conf`
- Ubuntu uses socket activation for ssh (`ssh.socket`)
- Private key never leaves your machine

⚠ DANGER CHECK

- Disabling passwords before your key works locks you out
- Test a second session before closing the first one
- Run `sudo sshd -t` before restarting; a bad config stops the server

KEYS, CONFIG, HARDENING

```
ssh-keygen -t ed25519 -C "bob@laptop"      # ~/.ssh/id_ed25519(.pub)
ssh-copy-id bob@192.168.1.20               # install public key
ssh bob@192.168.1.20                       # no password prompt now

# ~/.ssh/config
# Host home
#     HostName 192.168.1.20
#     User bob
#     IdentityFile ~/.ssh/id_ed25519
ssh home                                   # short alias

# server: /etc/ssh/sshd_config.d/10-hardening.conf
#   PasswordAuthentication no
#   PermitRootLogin no
#   AllowUsers bob
sudo sshd -t && sudo systemctl restart ssh

ssh -L 8080:localhost:80 home              # local :8080 → server :80
ssh -J jump.example.com bob@10.0.0.5       # via a jump host
```

CODE EXPLAINED

- → `ed25519` — modern, short, fast key type
- → `ssh-copy-id` — appends to `~/.ssh/authorized_keys`
- → `Host` blocks — aliases and per-host settings
- → drop-in `.conf` — survives package updates
- → `-L` — forward a local port through SSH

OTHER DISTROS

- Debian/Fedora — same OpenSSH; Fedora service is `sshd`
- macOS — enable Remote Login; same `ssh` client and `~/.ssh/config`

PREFER / AVOID

- Avoid password SSH on internet-facing hosts · Prefer keys only
- Avoid editing the main `sshd_config` · Prefer `sshd_config.d/` drop-ins

SSH OPTIONS

OPTION	DOES
`-p 2222`	port
`-i key`	identity file
`-L l:host:r`	local forward
`-R r:host:l`	remote forward
`-J jump`	jump host
`-v`	debug output

PERMISSION RULES

PATH	MODE
`~/.ssh`	700
`~/.ssh/authorized_keys`	600
`~/.ssh/id_ed25519`	600
`~/.ssh/id_ed25519.pub`	644
`~/.ssh/config`	600

⚠ COMMON MISTAKES

- Wrong permissions on `~/.ssh`: server silently ignores your key
- Copying the private key to the server instead of the `.pub`
- Restarting ssh with a typo in the config and locking yourself out

✓ TIPS & BEST PRACTICES

- Use `ssh-agent` (or the macOS keychain) so you type the passphrase once
- Debug logins with `ssh -v` and `journalctl -u ssh`
- Add fail2ban for brute-force protection (sheet 34)

QUICK SUMMARY

- `ssh-keygen` + `ssh-copy-id` — key login
- `~/.ssh/config` — aliases
- `PasswordAuthentication no` — harden
- `-L` / `-J` — tunnels and jump hosts

File Transfer

DESCRIPTION

Copy files between machines over SSH. **scp** is quick for single files. **rsync** is the workhorse: it only sends what changed, can resume, preserves permissions, and can mirror a directory with `--delete`. **sftp** gives an interactive session, and most GUI clients (FileZilla, Cyberduck) speak it too.

NOTES

- All three use your SSH keys and `~/.ssh/config`
- Trailing slash matters in rsync: `src/` vs `src`
- `--delete` removes files missing from the source
- `-n` (dry run) shows what would happen

COPY AND SYNC

```
scp report.pdf home:~/docs/              # laptop → server
scp home:/var/log/syslog ./              # server → laptop
scp -r photos/ home:/srv/media/          # directory

rsync -avz ~/projects/site/ home:/srv/site/       # contents of site
rsync -avzn --delete ./site/ home:/srv/site/      # dry run first!
rsync -avz --delete ./site/ home:/srv/site/       # mirror
rsync -avP big.iso home:~/                        # progress + resume
rsync -av --exclude='node_modules' app/ home:app/

sftp home                                # interactive
# sftp> put file.txt   get remote.txt   ls   cd   exit
wget -c https://releases.ubuntu.com/26.04/ubuntu.iso   # resume download
```

CODE EXPLAINED

- → `scp src dst` — `host:path` for the remote side
- → `-a` — archive: perms, times, links, recursive
- → `site/` — copy contents; `site` — copy the folder
- → `-n --delete` — preview a mirror safely
- → `-P` — progress + keep partial files

OTHER DISTROS

- Debian/Fedora — same tools
- macOS — `scp` / `sftp` built in; newer `rsync` via Homebrew

PREFER / AVOID

- Avoid `scp` for big or repeated transfers · Prefer `rsync`
- Avoid `--delete` without a dry run · Prefer `-n` first

RSYNC FLAGS

FLAG	MEANING
`-a`	archive mode
`-v`	verbose
`-z`	compress in transit
`-P`	progress + partial
`-n`	dry run
`--delete`	remove extra files at dest
`--exclude=PAT`	skip matches

TRAILING SLASH

COMMAND	RESULT
`rsync -a src/ dst/`	dst/file1, dst/file2
`rsync -a src dst/`	dst/src/file1
`rsync -a src/ dst`	same as first

⚠ COMMON MISTAKES

- Mixing up source and destination with `--delete`: wipes the original
- Forgetting the trailing slash and getting `site/site/`
- Copying as root and leaving root-owned files in user dirs

✓ TIPS & BEST PRACTICES

- Use `rsync -e 'ssh -p 2222'` for non-standard ports
- Add `--backup --backup-dir=../old` to keep replaced files
- Schedule rsync backups with a systemd timer (sheet 23)

QUICK SUMMARY

- `scp` — quick one-off copies
- `rsync -avz` — efficient sync
- `-n` before `--delete` — safety
- `sftp` — interactive / GUI clients

UFW Firewall

DESCRIPTION

UFW (Uncomplicated Firewall) is Ubuntu's friendly front end to the kernel firewall (nftables). It's installed but **inactive** by default. The usual setup: deny incoming, allow outgoing, allow SSH, then open only the ports your services need, optionally limited to your LAN.

NOTES

- Rules apply to IPv4 and IPv6
- App profiles live in `/etc/ufw/applications.d/`
- `limit` rate-limits repeated connections (SSH)
- Docker can bypass UFW rules (sheet 38)

⚠ DANGER CHECK

- `ufw enable` without an SSH rule cuts off remote sessions
- Always `ufw allow OpenSSH` (or your port) first
- `ufw reset` deletes all rules and disables the firewall

BASELINE SERVER FIREWALL

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH                    # or: sudo ufw limit 22/tcp
sudo ufw enable                           # turn it on
sudo ufw status verbose

sudo ufw allow 80,443/tcp                 # web server
sudo ufw allow from 192.168.1.0/24 to any port 445  # LAN
sudo ufw allow in on wg0                  # all from VPN
sudo ufw deny from 203.0.113.50           # block one address
sudo ufw app list                         # Nginx Full, OpenSSH, ...
sudo ufw allow "Nginx Full"

sudo ufw status numbered
sudo ufw delete 4                         # remove rule #4
sudo ufw logging on                       # log blocked packets
```

CODE EXPLAINED

- → defaults — block in, allow out
- → `allow OpenSSH` — app profile for port 22
- → `from <net> to any port` — LAN-only rule
- → `status numbered` + `delete N` — remove rules
- → `logging on` — blocks appear in the kernel log

OTHER DISTROS

- Debian — `ufw` available, nftables underneath
- Fedora/RHEL — `firewalld` with `firewall-cmd --add-service=`
- macOS — Application Firewall + `pf`

PREFER / AVOID

- Avoid opening ports "just in case" · Prefer allowing only what runs
- Avoid exposing admin services publicly · Prefer `from <LAN/VPN>` rules

RULE SYNTAX

RULE	MEANING
`allow 22/tcp`	port + protocol
`allow 8000:8010/tcp`	range
`allow from 10.0.0.5`	one source
`limit OpenSSH`	rate-limit
`deny out 25`	block outbound SMTP
`insert 1 deny from X`	rule at top

COMMON PORTS

PORT	SERVICE
`22`	SSH
`80 / 443`	HTTP / HTTPS
`445`	Samba
`2049`	NFS
`51820/udp`	WireGuard
`5432 / 3306`	PostgreSQL / MySQL (LAN only!)

⚠ COMMON MISTAKES

- Enabling UFW over SSH before allowing SSH
- Believing UFW protects Docker-published ports (it often doesn't)
- Rule order: an earlier `allow` beats a later `deny`

✓ TIPS & BEST PRACTICES

- Use `ufw limit` on SSH to slow brute force
- Bind databases to `127.0.0.1` as well as firewalling them
- Keep a console path (VM console) when changing firewall rules

QUICK SUMMARY

- deny in / allow out — baseline
- `allow OpenSSH` before `enable`
- `from <net>` — restrict by source
- `status numbered` / `delete` — manage

fail2ban & SSH Hardening

DESCRIPTION

Any server on the internet sees constant SSH login attempts. **fail2ban** watches logs and **bans IPs** that fail too often, using firewall rules. Combine it with SSH hardening: **keys only**, no root login, an allow-list of users, and optionally access only over a VPN. Changing the port reduces log noise but isn't real security.

NOTES

- `apt install fail2ban`; the `sshd` jail is enabled by default
- Never edit `jail.conf`; override in `jail.local` or `jail.d/`
- fail2ban reads systemd journal on Ubuntu
- Whitelist your own IP with `ignoreip`

/ETC/FAIL2BAN/JAIL.LOCAL

```
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 5
ignoreip = 127.0.0.1/8 192.168.1.0/24
bantime.increment = true          # repeat offenders: longer bans

[sshd]
enabled = true
backend = systemd
```

CODE EXPLAINED

- → `bantime` / `findtime` / `maxretry` — ban policy
- → `ignoreip` — never ban your LAN
- → `bantime.increment` — escalating bans
- → `backend = systemd` — read the journal
- → `sshd -T` — print effective SSH settings

CHECK AND MANAGE

```
sudo systemctl enable --now fail2ban
sudo fail2ban-client status              # list jails
sudo fail2ban-client status sshd         # banned IPs, counts
sudo fail2ban-client set sshd unbanip 203.0.113.9
sudo sshd -T | grep -Ei 'passwordauth|permitroot|allowusers'
```

OTHER DISTROS

- Debian — same fail2ban; Fedora — fail2ban + firewalld action
- macOS — no fail2ban by default; Remote Login usually LAN-only

PREFER / AVOID

- Avoid relying on a custom SSH port · Prefer keys + fail2ban (+ VPN)
- Avoid editing `jail.conf` · Prefer `jail.local`

SSH HARDENING SETTINGS

SETTING	VALUE
PasswordAuthentication	no
PermitRootLogin	no
KbdInteractiveAuthentication	no
AllowUsers	bob deploy
MaxAuthTries	3
X11Forwarding	no

FAIL2BAN COMMANDS

COMMAND	DOES
fail2ban-client status	list jails
status sshd	jail details
set sshd banip IP	manual ban
set sshd unbanip IP	unban
fail2ban-regex	test a filter

⚠ COMMON MISTAKES

- Banning yourself from a café IP with no `ignoreip` or console access
- Editing `jail.conf`: overwritten on upgrade
- Port-changing SSH and forgetting to update UFW rules first

✓ TIPS & BEST PRACTICES

- Put admin SSH behind Tailscale or WireGuard and close port 22 publicly
- Add jails for nginx (`nginx-http-auth`, `nginx-botsearch`)
- Check `journalctl -u ssh | grep Failed` to see attack volume

QUICK SUMMARY

- fail2ban — ban IPs after failures
- `jail.local` — your overrides
- keys only, no root — SSH basics
- VPN — best way to hide SSH

AppArmor

DESCRIPTION

AppArmor is Ubuntu's mandatory access control system. Each **profile** limits what one program may read, write and execute, even if it runs as root. Profiles run in **enforce** mode (block and log) or **complain** mode (log only). Snaps, LXD and many services ship with profiles out of the box.

NOTES

- Enabled by default; Fedora/RHEL use SELinux instead
- Profiles live in `/etc/apparmor.d/`
- Denials show as `apparmor="DENIED"` in the kernel log
- `apparmor-utils` provides `aa-*` tools

CHECK AND ADJUST PROFILES

```
sudo aa-status                              # loaded profiles + modes
sudo aa-status | grep -A3 "enforce mode"
sudo journalctl -k | grep 'apparmor="DENIED"' | tail
# ... apparmor="DENIED" operation="open" profile="/usr/sbin/nginx"
#     name="/data/www/index.html" requested_mask="r"

sudo apt install apparmor-utils
sudo aa-complain /usr/sbin/nginx            # log only, while debugging
sudo aa-logprof                             # turn denials into rules
sudo aa-enforce /usr/sbin/nginx             # back to blocking
sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.nginx   # reload one
ls /etc/apparmor.d/local/                   # site-specific additions
```

CODE EXPLAINED

- → `aa-status` — which profiles, which mode
- → DENIED line — profile, operation, path
- → `aa-complain` — temporarily log instead of block
- → `aa-logprof` — interactive rule builder
- → `local/` — add rules without editing vendor profiles

OTHER DISTROS

- Debian — AppArmor enabled by default too
- Fedora/RHEL — SELinux: `getenforce`, `ausearch`, `setsebool`
- macOS — sandbox profiles + SIP

PREFER / AVOID

- Avoid disabling AppArmor to fix one app · Prefer a rule in `local/`
- Avoid leaving profiles in complain mode · Prefer enforce once fixed

MODES

MODE	BEHAVIOR
`enforce`	block + log violations
`complain`	allow + log
`unconfined`	no profile
`kill`	kill on violation (rare)

TOOLS

COMMAND	DOES
`aa-status`	summary
`aa-enforce / aa-complain`	switch mode
`aa-disable`	unload profile
`aa-logprof`	update from logs
`aa-genprof`	build a new profile
`apparmor_parser -r`	reload

⚠ COMMON MISTAKES

- Moving a service's data dir (e.g. MySQL to `/data`) and getting mysterious permission errors
- Disabling AppArmor system-wide instead of fixing one rule
- Editing vendor profiles that updates overwrite

✓ TIPS & BEST PRACTICES

- Check the kernel log for DENIED before chasing file permissions
- Add paths in `/etc/apparmor.d/local/usr.sbin.<prog>` then reload
- Snap apps' confinement is AppArmor too (`snap connections`)

QUICK SUMMARY

- profiles — per-program restrictions
- enforce vs complain — block vs log
- DENIED logs — `journalctl -k`
- `local/` overrides — safe customizing

Ubuntu Pro & Security Updates

DESCRIPTION

Ubuntu Pro extends security coverage: **ESM** patches for `universe` packages and for older LTS releases, **Livepatch** kernel fixes without rebooting, and compliance tooling (CIS benchmarks, FIPS). It's free for personal use on a handful of machines. Canonical publishes **Ubuntu Security Notices (USNs)** for every fix.

NOTES

- Free Pro token: ubuntu.com/pro (personal account)
- Without Pro, `universe` packages get community fixes only
- Livepatch covers many, not all, kernel CVEs
- `pro fix CVE-...` checks and applies a specific fix

ATTACH AND USE PRO

```
pro status                              # services and entitlements
sudo pro attach C1aBcDeFgH...  # token from ubuntu.com/pro
pro status                             # esm-apps, esm-infra: enabled
sudo pro enable livepatch
canonical-livepatch status

apt list --upgradable                   # pending updates
pro security-status              # coverage of installed pkgs
sudo pro fix CVE-2024-6387     # check/fix one CVE

sudo pro enable usg                     # Ubuntu Security Guide
sudo apt install usg && sudo usg audit cis_level1_server
sudo pro detach                         # remove from Pro
```

CODE EXPLAINED

- → `pro attach` — link machine to your subscription
- → esm-infra / esm-apps — main vs universe coverage
- → Livepatch — kernel fixes without reboot
- → `pro fix CVE` — targeted check and patch
- → USG — CIS audit and hardening

OTHER DISTROS

- Debian — LTS/ELTS via Debian LTS and Freexian
- RHEL — subscription + kpatch; Fedora — short life, no ESM
- macOS — Rapid Security Responses

PREFER / AVOID

- Avoid running old LTS releases without ESM · Prefer Pro or upgrading
- Avoid ignoring USNs for internet-facing services · Prefer watching them

PRO SERVICES

SERVICE	GIVES
esm-infra	main/restricted fixes beyond 5 years
esm-apps	universe security fixes
livepatch	rebootless kernel fixes
usg	CIS / DISA-STIG tooling
fips	FIPS-certified crypto

STAY INFORMED

SOURCE	WHAT
ubuntu.com/security/notices	USNs
pro security-status	this machine
ubuntu-security-announce	mailing list
landscape	fleet management

⚠ COMMON MISTAKES

- Assuming unattended-upgrades patches `universe` without ESM Apps
- Treating Livepatch as a reason never to reboot
- Sharing a Pro token publicly (scripts, repos)

✓ TIPS & BEST PRACTICES

- Attach your home server and laptop to the free personal Pro tier
- Still reboot during maintenance windows to load new kernels fully
- Use `pro fix` during incident response for named CVEs

QUICK SUMMARY

- Ubuntu Pro — ESM, Livepatch, compliance
- free for personal use
- `pro status` / `security-status` — coverage
- USNs — track security fixes

Nginx & HTTPS

DESCRIPTION

Nginx is the usual web server and **reverse proxy** on Ubuntu: it serves static files and forwards requests to apps on local ports (FastAPI, Node, Docker containers). Each site is a **server block** in `sites-available`, enabled by a symlink. **Certbot** gets free Let's Encrypt certificates and renews them automatically.

NOTES

- Config test before reload: `sudo nginx -t`
- `reload` applies config without dropping connections
- Certbot renews via a systemd timer
- Apache: `a2ensite`, `a2enmod` play the same roles

/ETC/NGINX/SITES-AVAILABLE/TODO

```
server {
    listen 80;
    server_name todo.example.com;

    location /static/ {
        alias /srv/todo/static/;
        expires 7d;
    }
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

CODE EXPLAINED

- → `server_name` — which hostname this block serves
- → `location /static/` + `alias` — files from disk
- → `proxy_pass` — forward to the app
- → `X-Forwarded-*` — tell the app the real client/scheme
- → `certbot --nginx` — edits the block for TLS

ENABLE + HTTPS

```
sudo ln -s /etc/nginx/sites-available/todo /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d todo.example.com      # cert + HTTPS redirect
sudo certbot renew --dry-run
```

OTHER DISTROS

- Debian — identical layout
- Fedora/RHEL — single `/etc/nginx/conf.d/*.conf`, SELinux booleans for proxying
- macOS — `brew install nginx`, `/opt/homebrew/etc/nginx/`

PREFER / AVOID

- Avoid editing `nginx.conf` for each site · Prefer one file per site
- Avoid exposing app ports publicly · Prefer apps on `127.0.0.1` behind Nginx

USEFUL DIRECTIVES

DIRECTIVE	PURPOSE
`client_max_body_size 50M`	upload limit
`try_files $uri /index.html`	SPA routing
`return 301 https://$host$request_uri`	redirect to HTTPS
`gzip on`	compression
`add_header X-Frame-Options DENY`	security header

FILES & COMMANDS

ITEM	WHAT
`sites-available/`	all site configs
`sites-enabled/`	symlinks = active
`nginx -t`	test config
`/var/log/nginx/`	access + error logs
`certbot certificates`	list certs

⚠ COMMON MISTAKES

- Forgetting the trailing slash rules in `alias` and `proxy_pass`
- Leaving the `default` site enabled and catching all hostnames
- Port 80 blocked by UFW: Certbot's HTTP challenge fails

✓ TIPS & BEST PRACTICES

- `sudo ufw allow "Nginx Full"` opens 80 and 443
- Use Caddy if you want automatic HTTPS with less config
- Read `/var/log/nginx/error.log` first when you see 502

QUICK SUMMARY

- server block per site — sites-available + symlink
- `proxy_pass` — reverse proxy to apps
- `nginx -t` → `reload` — safe changes
- Certbot — free HTTPS, auto-renew

Docker on Ubuntu

Docker apt repo vs docker.io vs snap · docker group · Compose plugin · enable on boot · UFW caveat

DESCRIPTION

Install Docker from **Docker's official apt repository** for the newest Engine and the **Compose plugin**; Ubuntu's `docker.io` package and the snap also work but lag behind or behave differently. Add yourself to the `docker` group to skip `sudo`, and remember that published container ports **bypass UFW** by default.

NOTES

- Three options: Docker repo, `docker.io` (Ubuntu), snap
- `docker` group membership = root-equivalent
- Compose v2: `docker compose` (space, not hyphen)
- See the Docker set for images, volumes, Compose files

INSTALL FROM DOCKER'S REPO

```
sudo apt remove docker.io docker-compose podman-docker 2>/dev/null
# add key + repo (see sheet 15 for the pattern)
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
     -o /etc/apt/keyrings/docker.asc
# ... add docker.list with signed-by, then:
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io \
     docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker $USER && newgrp docker
docker run --rm hello-world
docker compose version

# publish only on localhost; let Nginx/UFW face the internet
docker run -d -p 127.0.0.1:8080:80 nginx
```

CODE EXPLAINED

- → remove old packages — avoid conflicts
- → Docker repo — current Engine + plugins
- → `enable --now` — start at boot
- → `usermod -aG docker` + `newgrp` — no sudo needed
- → `-p 127.0.0.1:8080:80` — not exposed publicly

OTHER DISTROS

- Debian — same Docker repo steps (debian path)
- Fedora — Docker repo or Podman (default, daemonless)
- macOS — Docker Desktop, OrbStack or Colima

PREFER / AVOID

- Avoid `-p 8080:80` on public servers · Prefer `127.0.0.1:` + reverse proxy
- Avoid mixing snap and apt Docker · Prefer one install method

INSTALL OPTIONS

SOURCE	NOTES
`Docker apt repo`	latest, recommended
`docker.io (Ubuntu)`	older, Ubuntu-maintained
`snap install docker`	confined; paths differ
`Podman`	rootless, Docker-compatible CLI

UBUNTU INTEGRATION

ITEM	WHERE
`data`	/var/lib/docker
`daemon config`	/etc/docker/daemon.json
`logs`	journalctl -u docker
`log size`	"log-opts": {"max-size": "10m"}
`UFW bypass`	Docker writes its own iptables rules

⚠ COMMON MISTAKES

- Expecting UFW to block `-p 0.0.0.0:5432:5432`: it won't
- Filling `/var` with images; prune with `docker system prune`
- Unbounded container logs eating the disk

✓ TIPS & BEST PRACTICES

- Set default log rotation in `/etc/docker/daemon.json`
- Move `/var/lib/docker` to a big disk with `data-root`
- Use rootless Docker or Podman for shared machines

QUICK SUMMARY

- Docker apt repo — recommended install
- `docker` group — convenience, root-equivalent
- `docker compose` — v2 plugin
- bind ports to 127.0.0.1 — UFW won't save you

Sheet 39

File Sharing: Samba & NFS

smb.conf shares · smbpasswd -a · access from macOS/Windows · nfs-kernel-server · /etc/exports · exportfs -ra

DESCRIPTION

Share folders on your network with **Samba** (SMB, the protocol Windows and macOS speak natively) or **NFS** (fast, simple, best between Linux machines). Samba users need a separate Samba password; NFS trusts client UIDs, so restrict exports to your LAN.

NOTES

- Samba ports: 445/tcp; NFS: 2049/tcp
- Samba users must exist as Linux users
- macOS: Finder → Go → Connect to Server → `smb://host/share`
- NFS v4 needs only port 2049

/ETC/SAMBA/SMB.CONF (ADD AT END)

```
[media]
   path = /srv/media
   browseable = yes
   read only = no
   valid users = bob @family
   create mask = 0664
   directory mask = 2775
```

CODE EXPLAINED

- → `[media]` — share name clients see
- → `valid users` — users and `@groups` allowed
- → `smbpasswd -a` — add Samba credential
- → `testparm` — validate config
- → `/etc/exports` + `exportfs -ra` — NFS share

SAMBA + NFS SETUP

```
sudo apt install samba
sudo smbpasswd -a bob                    # Samba password for bob
testparm                                 # check smb.conf
sudo systemctl restart smbd
sudo ufw allow from 192.168.1.0/24 to any app Samba

sudo apt install nfs-kernel-server
echo "/srv/media 192.168.1.0/24(rw,sync,no_subtree_check)" \
  | sudo tee -a /etc/exports
sudo exportfs -ra && sudo exportfs -v
# client: sudo mount -t nfs server:/srv/media /mnt/media
```

OTHER DISTROS

- Debian — same packages; Fedora — `samba`, `nfs-utils` + firewalld + SELinux booleans
- macOS — File Sharing in System Settings (SMB)

PREFER / AVOID

- Avoid guest-writable shares · Prefer `valid users`
- Avoid NFS exports to `*` · Prefer your LAN subnet

SAMBA SHARE OPTIONS

OPTION	EFFECT
`read only = no`	allow writes
`valid users`	who may connect
`force group`	files get this group
`create mask`	new file perms
`guest ok`	allow without password
`vfs objects = fruit`	better macOS support

NFS EXPORT OPTIONS

OPTION	EFFECT
`rw / ro`	read-write / read-only
`sync`	safe writes
`root_squash`	client root → nobody (default)
`no_subtree_check`	fewer problems
`all_squash`	everyone → anon user

⚠ COMMON MISTAKES

- Forgetting `smbpasswd -a`: Linux password alone doesn't work
- Linux perms block access even though Samba allows it
- `no_root_squash` on NFS: remote root becomes local root

✓ TIPS & BEST PRACTICES

- Add `vfs objects = catia fruit streams_xattr` for Time Machine/macOS
- Mount NFS in fstab with `_netdev,nofail,x-systemd.automount`
- Use `smbstatus` to see connected clients

QUICK SUMMARY

- Samba — SMB for Windows/macOS
- `smbpasswd -a` — Samba users
- NFS — `/etc/exports` + `exportfs -ra`
- limit both to your LAN

Sheet 40

VMs & System Containers

Multipass · LXD (lxc launch) · KVM/QEMU · virt-manager · kvm-ok · when to use which

DESCRIPTION

Ubuntu has several ways to run more Ubuntu. **Multipass** spins up Ubuntu VMs in one command, great for testing. **LXD** runs **system containers** (full OS, shared kernel, very light) and VMs, with snapshots and networking built in. **KVM/QEMU** with libvirt and `virt-manager` runs any OS, including Windows.

NOTES

- `kvm-ok` (cpu-checker) confirms hardware virtualization
- Multipass and LXD install as snaps
- LXD needs `lxd init` once
- Docker = app containers; LXD = whole-OS containers

THREE WAYS TO GET A TEST BOX

```
multipass launch 26.04 -n dev --cpus 2 --memory 4G --disk 20G
multipass shell dev
multipass list && multipass delete --purge dev

sudo snap install lxd && sudo lxd init --minimal
lxc launch ubuntu:26.04 web1                # system container
lxc exec web1 -- bash
lxc launch ubuntu:26.04 vm1 --vm            # full VM instead
lxc snapshot web1 before-upgrade
lxc restore web1 before-upgrade
lxc list

sudo apt install cpu-checker && kvm-ok   # "can be used"
sudo apt install qemu-kvm libvirt-daemon-system virt-manager
sudo usermod -aG libvirt $USER
```

CODE EXPLAINED

- → `multipass launch` — VM with CPU/RAM/disk sizes
- → `lxd init --minimal` — default storage + bridge
- → `lxc launch` — container; `--vm` — VM
- → `lxc snapshot` / `restore` — instant rollback
- → libvirt + virt-manager — GUI for any OS

OTHER DISTROS

- Debian — LXD/Incus, libvirt; Fedora — libvirt + GNOME Boxes
- macOS — UTM, Parallels; Multipass also runs on macOS

PREFER / AVOID

- Avoid testing upgrades on your real server · Prefer an LXD clone or snapshot
- Avoid full VMs for Linux services · Prefer LXD containers (lighter)

WHICH ONE?

NEED	USE
quick Ubuntu VM	Multipass
many light Linux systems	LXD containers
Windows / other OS	KVM + virt-manager
packaged apps	Docker (38)
home lab cluster	LXD cluster / Proxmox

LXC COMMANDS

COMMAND	DOES
`lxc launch img name`	create + start
`lxc exec name -- cmd`	run inside
`lxc stop / start / delete`	lifecycle
`lxc file push f name/root/`	copy in
`lxc config set name limits.memory 2GiB`	limits

⚠ COMMON MISTAKES

- Running VMs without KVM acceleration: painfully slow
- LXD and Docker fighting over iptables/forwarding rules
- Forgetting `--purge`: deleted Multipass instances keep disk space

✓ TIPS & BEST PRACTICES

- `multipass mount ~/code dev:/code` shares folders
- Use cloud-init with Multipass/LXD for repeatable setups (sheet 41)
- Incus is the community fork of LXD with the same `incus` CLI ideas

QUICK SUMMARY

- Multipass — one-command Ubuntu VMs
- LXD — system containers + VMs
- KVM/libvirt — any OS
- snapshots — safe experiments

cloud-init & Autoinstall

DESCRIPTION

cloud-init configures a machine on its first boot from a YAML file: users and SSH keys, packages, files and commands. Every Ubuntu cloud image, Multipass VM and LXD instance supports it. **Autoinstall** uses a similar YAML to install Ubuntu Server from the ISO without answering any prompts, so your servers are repeatable.

NOTES

- File must start with `#cloud-config`
- Runs once per instance (first boot)
- Logs: `/var/log/cloud-init-output.log`
- Autoinstall config goes under an `autoinstall:` key

USER-DATA (#CLOUD-CONFIG)

```
#cloud-config
hostname: web1
timezone: America/Chicago
users:
  - name: bob
    groups: [sudo, docker]
    shell: /bin/bash
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    ssh_authorized_keys:
      - ssh-ed25519 AAAAC3Nza... bob@laptop
package_update: true
packages: [nginx, fail2ban, htop]
write_files:
  - path: /etc/motd
    content: "Managed by cloud-init\n"
runcmd:
  - ufw allow OpenSSH && ufw --force enable
```

CODE EXPLAINED

- → `users:` — accounts, groups, SSH keys
- → `packages:` — installed on first boot
- → `write_files:` — drop config files
- → `runcmd:` — shell commands, run last
- → `schema` — validate before booting

OTHER DISTROS

- Debian/Fedora — cloud images also use cloud-init
- Installers: Debian preseed, Fedora/RHEL Kickstart
- macOS — MDM profiles instead

USE AND DEBUG

```
multipass launch 26.04 -n web1 --cloud-init user-data.yaml
cloud-init status --wait          # status: done
cloud-init schema --config-file user-data.yaml   # validate
```

PREFER / AVOID

- Avoid clicking through installs for every server · Prefer autoinstall/cloud-init YAML
- Avoid `NOPASSWD:ALL` on long-lived servers · Prefer password sudo after setup

COMMON MODULES

KEY	DOES
`users / groups`	accounts
`ssh_authorized_keys`	keys for default user
`packages / package_upgrade`	apt
`write_files`	create files
`runcmd / bootcmd`	commands (late / early)
`snap: commands:`	install snaps

AUTOINSTALL SECTIONS

KEY	SETS
`identity`	hostname, user, password hash
`storage: layout: lvm`	disk layout
`ssh: install-server`	OpenSSH
`packages`	extra packages
`late-commands`	final shell commands
`user-data`	embedded cloud-init

⚠ COMMON MISTAKES

- Missing `#cloud-config` first line: file ignored
- YAML indentation errors: whole config skipped silently
- Expecting changes to apply on reboot: cloud-init runs once per instance

✓ TIPS & BEST PRACTICES

- Use `cloud-init clean --logs --reboot` to re-run while testing
- Serve autoinstall from a USB `CIDATA` volume or HTTP
- Keep the YAML in git as documentation of your servers

QUICK SUMMARY

- `#cloud-config` — first-boot setup
- users, packages, files, runcmd — main modules
- autoinstall — unattended Server installs
- `cloud-init status` / logs — debug

Sheet 42

GNOME Desktop & Settings

GNOME Shell · gsettings · dconf · extensions · Wayland vs X11 · shortcuts · .desktop launchers

DESCRIPTION

Ubuntu Desktop runs **GNOME** with Ubuntu's own touches (dock, app indicators). Most settings live in **dconf**, which you can script with **gsettings**. **Extensions** add features like tiling or system monitors. Ubuntu uses **Wayland** by default; X11 sessions remain available for old apps and some remote-desktop tools.

NOTES

- `Super` key opens Activities
- `echo $XDG_SESSION_TYPE` → `wayland` or `x11`
- App launchers are `.desktop` files
- Tweaks app: `sudo apt install gnome-tweaks`

SCRIPT YOUR DESKTOP

```
gsettings get org.gnome.desktop.interface color-scheme
gsettings set org.gnome.desktop.interface \
    color-scheme 'prefer-dark'
gsettings set org.gnome.shell.extensions.dash-to-dock \
    dock-position 'BOTTOM'
gsettings set org.gnome.desktop.peripherals.touchpad \
    tap-to-click true
gsettings list-recursively org.gnome.desktop.wm.keybindings | head

dconf dump /org/gnome/ > gnome-settings.ini     # back up
dconf load /org/gnome/ < gnome-settings.ini     # restore

sudo apt install gnome-shell-extension-manager
gnome-extensions list --enabled
echo $XDG_SESSION_TYPE                          # wayland
```

CODE EXPLAINED

- → `gsettings get/set schema key` — read/write a setting
- → `color-scheme 'prefer-dark'` — dark mode
- → `dconf dump` / `load` — back up all GNOME settings
- → Extension Manager — browse and install extensions
- → `XDG_SESSION_TYPE` — Wayland or X11

OTHER DISTROS

- Fedora — vanilla GNOME, same gsettings
- Kubuntu — KDE Plasma, settings via `kwriteconfig6`
- macOS — `defaults write` is the gsettings equivalent

PREFER / AVOID

- Avoid editing dconf binary files · Prefer `gsettings` / `dconf load`
- Avoid many extensions after upgrades · Prefer a few maintained ones

HANDY SHORTCUTS

KEYS	DOES
Super	Activities / search
Super+A	app grid
Super+←/→	snap window
Ctrl+Alt+T	terminal
Super+L	lock
Shift+PrtSc / PrtSc	screenshot tool

LAUNCHER (.DESKTOP)

LINE	MEANING
[Desktop Entry]	header
Name=My App	label
Exec=/opt/app/run.sh	command
Icon=/opt/app/icon.png	icon
Type=Application	type
~/.local/share/applications/	where to save

⚠ COMMON MISTAKES

- Extensions breaking after a GNOME version bump: disable and update
- Screen sharing tools that only support X11 failing on Wayland
- Hand-copying `.config` dirs between versions instead of `dconf dump`

✓ TIPS & BEST PRACTICES

- Keep `dconf dump` output in your dotfiles repo
- Install `gnome-tweaks` for fonts and title-bar buttons
- Pick "Ubuntu on Xorg" at the login gear icon if an app needs X11

QUICK SUMMARY

- GNOME — Ubuntu's desktop
- `gsettings` — script settings
- `dconf dump/load` — back up / restore
- Wayland default; X11 optional

Drivers & Hardware

DESCRIPTION

Most hardware works out of the box because drivers are in the kernel. For proprietary drivers (NVIDIA GPUs, some Wi-Fi chips), **`ubuntu-drivers`** detects the hardware and installs the recommended package. **`lspci`**, **`lsusb`** and **`lshw`** identify devices, and **`fwupdmgr`** updates firmware from the LVFS.

NOTES

- Secure Boot may ask you to enroll a key (MOK) for drivers
- Desktop GUI: Software & Updates → Additional Drivers
- `-server` NVIDIA drivers exist for headless machines
- Old MacBooks often need `broadcom-sta-dkms` for Wi-Fi

IDENTIFY AND INSTALL

```
lspci -nnk | grep -A3 -i vga        # GPU + driver in use
lsusb                               # USB devices
sudo lshw -short                    # full hardware summary
inxi -Fxz                           # readable overview

ubuntu-drivers devices              # detected + recommended
sudo ubuntu-drivers install         # install recommended drivers
sudo ubuntu-drivers install nvidia:570  # pick version
nvidia-smi                          # GPU status after reboot

fwupdmgr get-devices
fwupdmgr refresh && fwupdmgr get-updates
sudo fwupdmgr update                # BIOS/firmware updates

bluetoothctl                        # power on, scan on
lpstat -p                           # printers (CUPS)
```

CODE EXPLAINED

- → `lspci -nnk` — device IDs + driver in use
- → `ubuntu-drivers devices` — what's recommended
- → `ubuntu-drivers install` — install them
- → `nvidia-smi` — confirms the NVIDIA driver works
- → `fwupdmgr` — vendor firmware updates

OTHER DISTROS

- Debian — non-free-firmware repo, `nvidia-detect`
- Fedora — RPM Fusion for NVIDIA (akmod-nvidia)
- macOS — drivers ship with the OS

PREFER / AVOID

- Avoid NVIDIA's `.run` installer · Prefer `ubuntu-drivers` packages
- Avoid random PPAs for drivers · Prefer Ubuntu's restricted repo

INFO COMMANDS

COMMAND	SHOWS
`lspci`	PCI devices
`lsusb`	USB devices
`lsblk`	disks
`lscpu`	CPU
`sensors`	temps (lm-sensors)
`dmidecode -t system`	model, serial, BIOS

FIXING HARDWARE

PROBLEM	TRY
`no Wi-Fi`	ubuntu-drivers devices
`black screen (NVIDIA)`	boot with nomodeset, reinstall driver
`sleep issues`	update firmware/BIOS
`no sound`	pavucontrol, check output
`printer`	localhost:631 (CUPS)

⚠ COMMON MISTAKES

- Skipping MOK enrollment after installing drivers with Secure Boot: driver won't load
- Mixing `.run` NVIDIA installs with packaged drivers
- Updating firmware on battery power

✓ TIPS & BEST PRACTICES

- Use `sudo dmesg | grep -i firmware` to spot missing firmware
- Search the hardware ID (`8086:a0f0`) for known issues
- Use the HWE kernel (`linux-generic-hwe-26.04`) for newer hardware

QUICK SUMMARY

- `lspci` / `lsusb` / `lshw` — identify
- `ubuntu-drivers` — proprietary drivers
- `fwupdmgr` — firmware updates
- MOK enroll — drivers with Secure Boot

Ubuntu on WSL

DESCRIPTION

WSL (Windows Subsystem for Linux) runs real Ubuntu inside Windows using a lightweight VM. You get the full Ubuntu userland, `apt`, and even **systemd**, with fast access to your Windows files and tools. Keep projects inside the Linux filesystem for speed, and use **VS Code's WSL extension** to edit them.

NOTES

- WSL 2 is the default (real Linux kernel)
- Windows drives appear under `/mnt/c`
- Linux files from Windows: `\\wsl$\Ubuntu`
- systemd is on by default in new Ubuntu WSL installs

POWERSHELL (WINDOWS) + UBUNTU

```
# PowerShell (admin)
wsl --install -d Ubuntu-26.04
wsl -l -v                          # distros + WSL version
wsl --update
wsl --shutdown                     # restart the VM

# inside Ubuntu
sudo apt update && sudo apt upgrade
cd ~ && git clone https://github.com/me/app.git   # Linux FS = fast
explorer.exe .                     # open this folder in Windows
code .                             # VS Code (WSL extension)
cat /etc/wsl.conf
# [boot]
# systemd=true
# [automount]
# options = "metadata"
```

CODE EXPLAINED

- → `wsl --install -d` — install a distro
- → `wsl -l -v` — list with versions
- → `wsl --shutdown` — apply config changes
- → `~/` projects — much faster than `/mnt/c`
- → `wsl.conf` — per-distro settings

OTHER DISTROS

- Debian, openSUSE, Fedora, Arch — also available on WSL
- macOS — no WSL; use Multipass, OrbStack, UTM

PREFER / AVOID

- Avoid working in `/mnt/c/...` · Prefer the Linux home directory
- Avoid Windows editors writing Linux files over `\\wsl$` heavily · Prefer VS Code WSL mode

WSL COMMANDS

COMMAND	DOES
`wsl -l -v`	list distros
`wsl -d Ubuntu`	start a distro
`wsl --set-default Ubuntu`	default distro
`wsl --export Ubuntu u.tar`	backup
`wsl --import`	restore/clone
`wsl --unregister Ubuntu`	delete (data gone)

CONFIG FILES

FILE	SCOPE
`/etc/wsl.conf`	one distro (systemd, automount, user)
`%UserProfile%\.wslconfig`	all distros (RAM, CPUs, networking)
`networkingMode=mirrored`	share Windows network
`memory=8GB`	VM memory cap

⚠ COMMON MISTAKES

- Slow builds because the repo lives on `/mnt/c`
- `wsl --unregister` without an export: everything is deleted
- Line-ending issues: set `git config core.autocrlf input`

✓ TIPS & BEST PRACTICES

- Use `wsl --export` before big changes
- Cap memory in `.wslconfig` if WSL eats RAM
- Docker Desktop integrates with WSL 2 distros

QUICK SUMMARY

- `wsl --install -d Ubuntu-26.04` — install
- WSL 2 + systemd — real Ubuntu
- keep code in `~` — speed
- VS Code WSL — edit seamlessly

System Info & Performance

DESCRIPTION

When a machine feels slow, check the four resources: **CPU**, **memory**, **disk space/IO** and **network**. **`uptime`** shows load average, **`free -h`** memory, **`df -h`** space, **`iostat`** disk activity, and **`btop`** all of it at once. Linux uses spare RAM for cache, so low "free" memory is normal: look at **available**.

NOTES

- Load average ≈ runnable tasks; compare to CPU count
- `available` in `free -h` is what apps can use
- Swap use alone isn't bad; constant swapping is
- `iostat` comes from the `sysstat` package

QUICK HEALTH CHECK

```
uptime                          # load average: 0.42, 0.55, 0.61
nproc                           # 4 CPUs → load 4 = fully busy
uname -a                        # kernel, arch
lscpu | grep "Model name"
free -h                         # look at "available"
df -h -x tmpfs -x squashfs      # real filesystems only
du -sh /var/* | sort -h         # biggest dirs in /var
sudo ncdu /                     # interactive disk usage
vmstat 2 5                      # CPU, memory, swap, IO every 2 s
iostat -xz 2                    # per-disk utilization (sysstat)
btop                            # all-in-one dashboard
systemd-cgtop                   # resource use per service
```

CODE EXPLAINED

- → load avg — 1, 5, 15 minute averages
- → `nproc` — compare load to core count
- → `df -h -x ...` — hide virtual filesystems
- → `du | sort -h` — find space hogs
- → `iostat -xz` — `%util` near 100 = disk bound

OTHER DISTROS

- Debian/Fedora — same tools
- macOS — `top`, `vm_stat`, `df -h`, Activity Monitor

PREFER / AVOID

- Avoid reading "free" memory as a problem · Prefer "available"
- Avoid guessing · Prefer measuring CPU, RAM, disk, network in turn

SYMPTOMS → TOOL

SYMPTOM	CHECK
slow everything	uptime, btop
out of memory	free -h, journalctl -k | grep -i oom
disk full	df -h, ncdu
disk slow	iostat -xz
network slow	mtr, iperf3
one service	systemd-cgtop

VMSTAT COLUMNS

COLUMN	MEANING
r	runnable processes
si / so	swap in / out
bi / bo	blocks in / out
us / sy	user / system CPU
wa	waiting on IO
id	idle

⚠ COMMON MISTAKES

- Deleting files that a running process holds: space isn't freed (`lsof +L1`)
- Ignoring inode exhaustion: `df -i` can be full while `df -h` isn't
- Treating squashfs snap mounts as full disks

✓ TIPS & BEST PRACTICES

- Find deleted-but-open files: `sudo lsof +L1`
- Enable `sysstat` history: `sudo systemctl enable --now sysstat`
- Add a swap file on small VMs: `fallocate` + `mkswap` + `swapon`

QUICK SUMMARY

- `uptime` / `nproc` — CPU load
- `free -h` available — memory
- `df` / `du` / `ncdu` — space
- `iostat` / `btop` — IO and overview

Boot, GRUB & Recovery

DESCRIPTION

GRUB is the boot menu that loads the Linux kernel. Its settings live in **/etc/default/grub** and take effect after **update-grub**. When a system won't boot, pick an older kernel or **recovery mode** from the GRUB menu, or boot a live USB and repair from there. `systemd-analyze` shows what slows boot down.

NOTES

- Hold Shift (BIOS) or tap Esc (UEFI) to show the menu
- Ubuntu keeps the previous kernel as a fallback
- Recovery mode → root shell: remount with `mount -o remount,rw /`
- Emergency mode usually means a bad `/etc/fstab`

⚠ DANGER CHECK

- A broken GRUB config can make the system unbootable
- Removing the running or only kernel leaves nothing to boot
- Have a live USB ready before editing boot settings

TUNE AND REPAIR BOOT

```
sudoedit /etc/default/grub
#   GRUB_TIMEOUT_STYLE=menu
#   GRUB_TIMEOUT=5
#   GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
sudo update-grub                          # apply

systemd-analyze                           # total boot time
systemd-analyze blame | head              # slowest units
dpkg -l 'linux-image-*' | grep ^ii        # installed kernels
uname -r                                  # running kernel

# recovery mode → "root" shell:
mount -o remount,rw /
nano /etc/fstab                           # fix the bad line
systemctl default                         # continue booting

# from a live USB (reinstall GRUB):
sudo mount /dev/nvme0n1p2 /mnt && sudo mount /dev/nvme0n1p1 /mnt/boot/efi
for d in dev proc sys; do sudo mount --rbind /$d /mnt/$d; done
sudo chroot /mnt grub-install && sudo chroot /mnt update-grub
```

CODE EXPLAINED

- → `/etc/default/grub` — menu and kernel options
- → `update-grub` — regenerate the boot config
- → `systemd-analyze blame` — slow services
- → recovery: remount `/` read-write to fix files
- → live USB + chroot — repair an unbootable install

OTHER DISTROS

- Debian — same GRUB; Fedora — `grub2-mkconfig`, BLS entries
- macOS — Recovery mode (Cmd+R / hold power on Apple silicon)

PREFER / AVOID

- Avoid editing `/boot/grub/grub.cfg` · Prefer `/etc/default/grub` + `update-grub`
- Avoid deleting old kernels by hand · Prefer `apt autoremove`

KERNEL PARAMETERS

PARAMETER	USE
`quiet splash`	graphical boot
`nomodeset`	GPU driver problems
`systemd.unit=rescue.target`	rescue mode
`init=/bin/bash`	last-resort shell
`fsck.mode=force`	check disks at boot

BOOT PROBLEMS

SYMPTOM	FIX
`emergency mode`	fix /etc/fstab
`black screen`	nomodeset, older kernel
`grub rescue>`	reinstall GRUB from live USB
`slow boot`	systemd-analyze blame
`new kernel breaks`	boot previous kernel, report bug

⚠ COMMON MISTAKES

- Forgetting `update-grub` after editing `/etc/default/grub`
- Purging the running kernel during cleanup
- Fixing fstab in recovery without remounting `/` read-write

✓ TIPS & BEST PRACTICES

- Set `GRUB_TIMEOUT_STYLE=menu` on servers with console access
- Use `nofail` in fstab for non-essential disks (sheet 25)
- Boot-Repair (live USB) automates many GRUB fixes

QUICK SUMMARY

- GRUB — boot menu, `/etc/default/grub`
- `update-grub` — apply changes
- recovery mode / older kernel — first aid
- live USB + chroot — deep repair

Backup & Restore

rsync backups · tar snapshots · Déjà Dup · Timeshift · ZFS/LVM snapshots · restic / borg · 3-2-1 · test restores

DESCRIPTION

A backup you haven't restored is a guess. Follow **3-2-1**: three copies, on two kinds of media, one **off-site**. On desktops, **Déjà Dup** (Backups app) covers your files and **Timeshift** snapshots the system. On servers, **restic** or **borg** give encrypted, deduplicated, versioned backups to disks or cloud storage.

NOTES

- Snapshots (ZFS/LVM) are not backups: same disk
- Back up `/etc`, `/home`, `/srv`, `/var/lib/<app>`, DB dumps
- Dump databases first (`pg_dump`), then back up the dump
- Encrypt anything that leaves the house

⚠ DANGER CHECK

- `rsync --delete` to a backup can mirror an accidental deletion
- Restoring over live files can overwrite newer data
- Restore to a temp directory first, then copy what you need

RESTIC TO AN EXTERNAL DISK

```
sudo apt install restic
export RESTIC_REPOSITORY=/mnt/backup/restic
export RESTIC_PASSWORD_FILE=/root/.restic-pass
restic init                                  # once
sudo -u postgres pg_dump app > /srv/dumps/app.sql
restic backup /etc /home /srv --exclude-caches \
       --exclude '/home/*/.cache'
restic snapshots                             # list versions
restic forget --keep-daily 7 --keep-weekly 4 \
       --keep-monthly 6 --prune              # retention
restic check                                 # verify repo
restic restore latest --target /tmp/restore \
       --include /etc/nginx                  # test restore!

# quick one-off: tar with date
sudo tar -czf /mnt/backup/etc-$(date +%F).tgz /etc
```

CODE EXPLAINED

- → `restic init` — create encrypted repo
- → DB dump first — consistent database backup
- → `--exclude-caches` — skip cache dirs
- → `forget --keep-* --prune` — retention policy
- → `restore --target /tmp` — safe test restore

OTHER DISTROS

- Debian/Fedora — same restic/borg; Fedora uses Btrfs snapshots + Timeshift/Snapper
- macOS — Time Machine; restic works too

PREFER / AVOID

- Avoid backups on the same disk only · Prefer external + off-site
- Avoid never testing restores · Prefer scheduled restore drills

TOOLS

TOOL	BEST FOR
Déjà Dup	desktop home folders
Timeshift	system snapshots (desktop)
restic	encrypted, cloud/S3 backends
borg / borgmatic	fast dedup to SSH targets
rsync	simple mirrors
zfs send	pool-to-pool replication

WHAT TO BACK UP

PATH	WHY
/etc	configuration
/home	user data
/srv, /var/www	sites and data
DB dumps	consistent databases
/var/lib/docker/volumes	container data
package list	apt-mark showmanual

⚠ COMMON MISTAKES

- Backing up live database files instead of dumps: often unusable
- Losing the restic/borg password: backups are unrecoverable
- Only snapshots on the same pool: disk failure takes both

✓ TIPS & BEST PRACTICES

- Run backups from a systemd timer and ping a health check (sheet 23)
- Store a copy of the repo password in a password manager
- Use B2/S3 or a friend's server as the off-site copy

QUICK SUMMARY

- 3-2-1 — three copies, two media, one off-site
- restic / borg — encrypted, versioned
- dump databases first
- test restores regularly

Sheet 48

Quick Reference — Files, Users & Packages

file commands · permissions · users & sudo · apt / dpkg / snap · archives · help

FILES

COMMAND	DOES	#
`ls -lah`	list all, sizes	06
`cp -a src dst`	copy, keep metadata	06
`mv a b`	move/rename	06
`mkdir -p a/b`	make dirs	06
`rm -i / trash-put`	safe delete	07
`less / tail -F`	read / follow	08
`find . -name '*.log'`	search	10

PERMISSIONS

COMMAND	DOES	#
`chmod 644 f / 755 d`	set mode	09
`chmod u+x f`	add execute	09
`chown user:grp f`	owner	09
`chmod 2775 dir`	setgid shared dir	13
`setfacl -m u:x:rwx`	ACL entry	13
`umask 027`	default perms	13

USERS & SUDO

COMMAND	DOES	#
`adduser alice`	new user	11
`usermod -aG grp alice`	add to group	11
`id / groups`	membership	11
`passwd -l alice`	lock	11
`sudo -i`	root shell	12
`visudo -f /etc/sudoers.d/x`	sudo rule	12

PACKAGES

COMMAND	DOES	#
`apt update && apt upgrade`	update	14
`apt install / purge`	add / remove	14
`apt policy pkg`	versions	14
`add-apt-repository ppa:x/y`	PPA	15
`apt install ./f.deb`	local .deb	16
`dpkg -S /path`	owning pkg	16
`snap install x --classic`	snap	17

ARCHIVES

COMMAND	DOES	#
`tar -czf a.tgz dir`	create	10
`tar -xf a.tgz -C dir`	extract	10
`tar -tf a.tgz`	list	10
`zip -r a.zip dir`	zip	10

HELP & INFO

COMMAND	DOES	#
`man / --help / tldr`	docs	04
`apropos word`	find command	04
`lsb_release -a`	Ubuntu version	01
`hostnamectl`	host info	03

⚠ COMMON MISTAKES

- `usermod -G` without `-a` removes other groups (11)
- `chmod -R 777` as a fix (09)
- `rm -rf` with an unchecked variable (07)

✓ TIPS & BEST PRACTICES

- Read `man` / `tldr` before running unfamiliar flags
- Use `sudoedit` and `visudo` for system files
- Prefer apt repos over random `.deb` downloads

QUICK SUMMARY

- files — `ls`, `cp -a`, `mv`, `find`
- permissions — `chmod`, `chown`, groups
- users — `adduser`, `usermod -aG`, `sudo`
- packages — `apt`, `dpkg`, `snap`

Sheet 49

Quick Reference — Services, Storage & Network

systemctl / journalctl · disks & mounts · ip / ss / netplan · SSH & rsync · UFW · config paths

SERVICES & LOGS

COMMAND	DOES	#
`systemctl status x`	state + logs	20
`systemctl enable --now x`	boot + start	20
`systemctl daemon-reload`	after unit edits	21
`systemctl edit x`	override	21
`journalctl -u x -f`	follow logs	22
`journalctl -b -1 -p err`	last boot errors	22
`systemctl list-timers`	schedules	23

STORAGE

COMMAND	DOES	#
`lsblk -f`	disks + filesystems	24
`parted /dev/sdX mklabel gpt`	partition table	24
`mkfs.ext4 -L name /dev/sdX1`	format	25
`mount -a / findmnt`	fstab test / view	25
`lvextend -r -l +100%FREE lv`	grow LV	26
`zpool status`	ZFS health	27
`df -h / ncdu`	space	45

NETWORK

COMMAND	DOES	#
`ip -br a / ip route`	addresses / gateway	28
`ss -tulpn`	listening ports	28
`netplan try`	safe apply	29
`resolvectl status`	DNS	30
`ssh-copy-id host`	key login	31
`rsync -avzn --delete a/ h:b/`	dry-run mirror	32

SECURITY

COMMAND	DOES	#
`ufw allow OpenSSH && ufw enable`	firewall	33
`ufw status numbered`	rules	33
`fail2ban-client status sshd`	bans	34
`aa-status`	AppArmor	35
`pro status`	Ubuntu Pro	36
`unattended-upgrade --dry-run`	auto updates	18

KEY CONFIG PATHS

PATH	CONFIGURES	#
`/etc/netplan/*.yaml`	network	29
`/etc/fstab`	mounts	25
`/etc/ssh/sshd_config.d/`	SSH server	31
`/etc/sudoers.d/`	sudo rules	12
`/etc/systemd/system/`	units	21
`/etc/default/grub`	boot	46

RECOVERY

COMMAND	DOES	#
`mount -o remount,rw /`	fix files in recovery	46
`update-grub`	regenerate boot	46
`dpkg --configure -a`	finish installs	16
`restic restore latest --target /tmp/r`	restore	47

⚠ COMMON MISTAKES

- `netplan apply` over SSH without `netplan try` (29)
- `ufw enable` before allowing SSH (33)
- Untested fstab edits before reboot (25)

✓ TIPS & BEST PRACTICES

- `systemctl status` + `journalctl -u` first, always
- Keep console access when touching network, SSH or firewall
- Snapshot or back up before risky changes

QUICK SUMMARY

- services — `systemctl`, `journalctl`
- storage — `lsblk`, `mount -a`, LVM, ZFS
- network — `ip`, `ss`, Netplan, SSH
- security — UFW, fail2ban, Pro